

UD 1 - PROGRAMACIÓN ORIENTADA A OBJETOS Y ELEMENTOS BÁSICOS DE JAVA

ENRIQUE RO



UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

Indice

- ▶ 1.- Principios de la programación orientada a objetos
- ▶ 2.- Notación UML
- ▶ 3.- Primer programa con NetBeans: HolaMundo
- ▶ 4.- Elementos básicos del lenguaje
 - ▶ 4.1.- Primera vuelta a la espiral
 - ▶ 4.2.- Segunda vuelta a la espiral
 - ▶ 4.3.- Tercera vuelta a la espiral

UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

1 - Principios de la programación O.O.

1.- Principios de la programación orientada a objetos

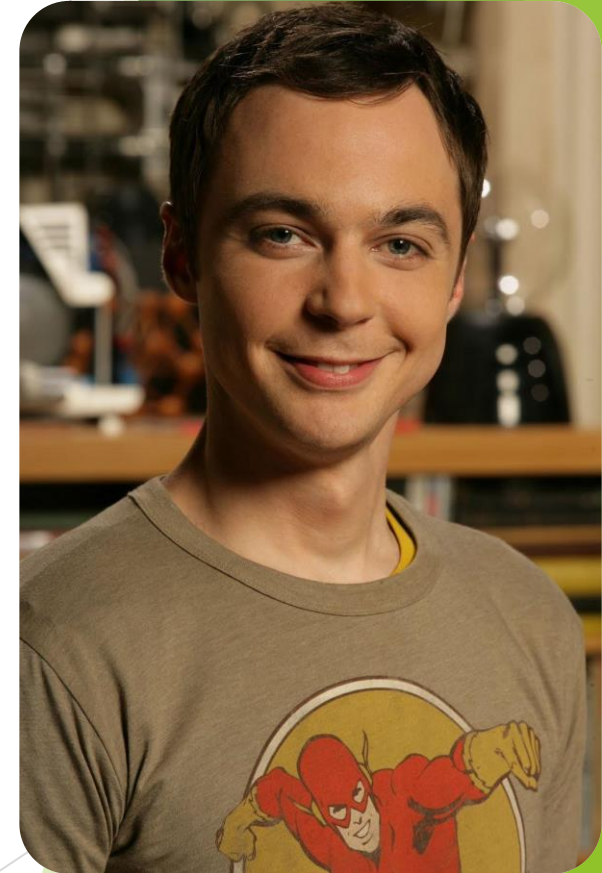
- ▶ Allá por los años 70 existían ordenadores con “cierta” potencia pero el software que se desarrollaba no era capaz de sacarle partido.
- ▶ Esto provocó la llamada “crisis del software” ya que se desarrollaba mal, no existían normas ni procedimientos y faltaba formación. Todo era demasiado nuevo. Esto provocaba que la finalización de un proyecto de software se retrasase en varias ocasiones, que todo saliese más caro y que la calidad del producto final fuera mediocre.



UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

1 - Principios de la programación O.O.

- ▶ El informático era una especie de “freak” que hablaba un “idioma” que no entendían los clientes que contrataban el desarrollo de software.
- ▶ Se detectó entonces una “**brecha comunicacional**” entre el lenguaje que manejaba una empresa (facturas, clientes...) y el lenguaje del informático (bucle, funciones, variables...). Había que hacer algo...
- ▶ La **ingeniería del software** viene poner orden en el caso y el paradigma o modelo de **Programación Orientada a Objetos** viene a mejorar la brecha comunicacional existente.
- ▶ Veamos los principios de este paradigma de programación.



UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

1 - Principios de la programación O.O.

- Si un lenguaje de programación es orientado a objetos entonces debe cumplir los siguientes principios:
 - *Todo es un objeto*
 - *Todo objeto es de una clase*
 - *Los objetos se comunican mediante mensajes*
 - *Una aplicación es una comunidad de objetos que interactúa para resolver un problema.*

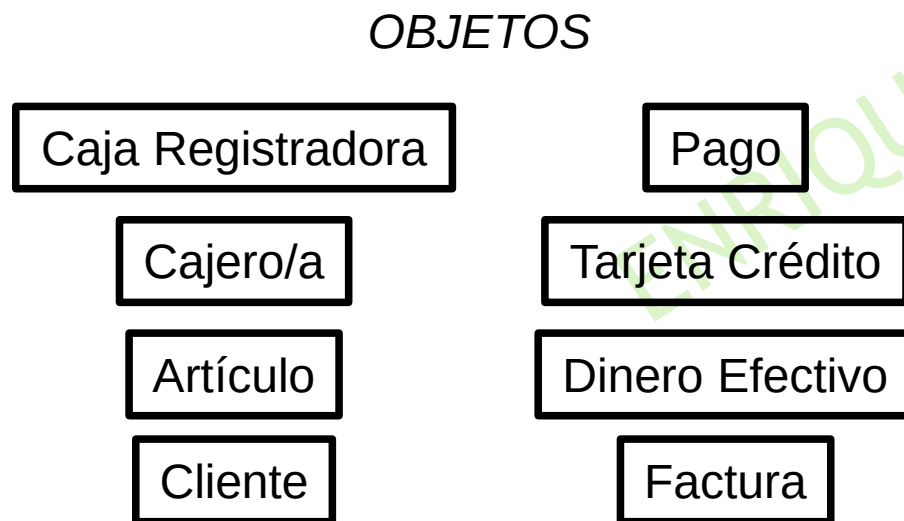
Analicemos cada uno de ellos...



UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

1 - Principios de la programación O.O.

- ▶ «Todo es un objeto»
- ▶ Cualquier problema puede ser modelado como un conjunto de objetos que interactúan entre ellos.



UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

1 - Principios de la programación O.O.

- Los objetos se caracterizan por sus:
 - **Propiedades o atributos:** *color, peso, nombre, apellidos, precio, altura...*
 - **Comportamientos o acciones que podemos realizar con ellos:** *arrancar, parar, saltar, comer, abrir, cerrar, pagar, cobrar...*

Propiedades: color, textura, cantidad de aire...



Comportamientos: botar, rodar, girar...

UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

1 - Principios de la programación O.O.

► OBJETO MECHERO

- **Propiedades:** *alto, ancho, profundo, estado (encendido/apagado), mililitros de gas...*
- **Comportamientos:** *encender durante X segundos, apagar, recargar X mililitros de gas...*

OJO: cuando programemos un objeto seremos los responsables de que el objeto siempre quede en un **estado coherente**. No tiene sentido que el mechero se encienda si no tiene gas...

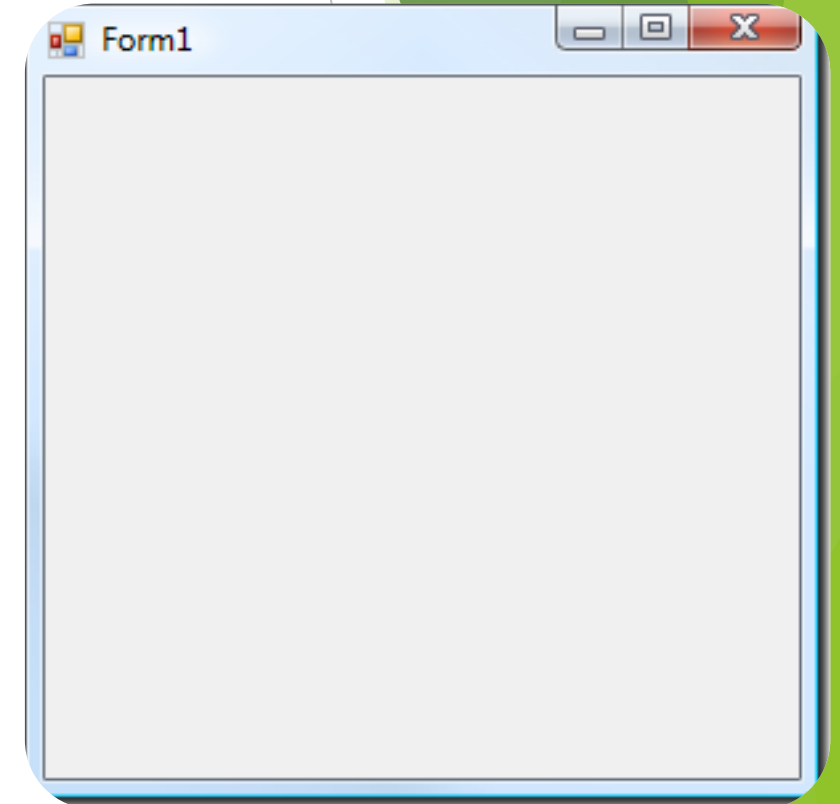


UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

1 - Principios de la programación O.O.

► OBJETO VENTANA DE ESCRITORIO

- **Propiedades:** *tamaño horizontal, tamaño vertical, color, visible...*
- **Comportamientos:** *minimizar, maximizar, redimensionar, abrir, cerrar...*



UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

1 - Principios de la programación O.O.

- Navega a esta web y resuelve el problema arrastrando acciones sobre el espacio de trabajo

<https://studio.code.org/s/mc/stage/1/puzzle/3>

- Piensa las propiedades y comportamientos del objeto hombre (el muñeco del juego):
 - Propiedades: ¿?
 - Comportamientos: ¿?





Ejecutar

Instrucciones

¡Hora de esquila! Usa el comando "esquila" para obtener lana de ambas ovejas.

OK

Bloques

avanzar

girar a la izquierda ↶

girar a la derecha ↷

trasquila

al ejecutar

Espacio de trabajo: 1 / 7 bloques

Volver a empezar

Mostrar el código

UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

1 - Principios de la programación O.O.

- Navega a esta web y resuelve el problema arrastrando acciones sobre el espacio de trabajo

<https://studio.code.org/s/mc/stage/1/puzzle/3>

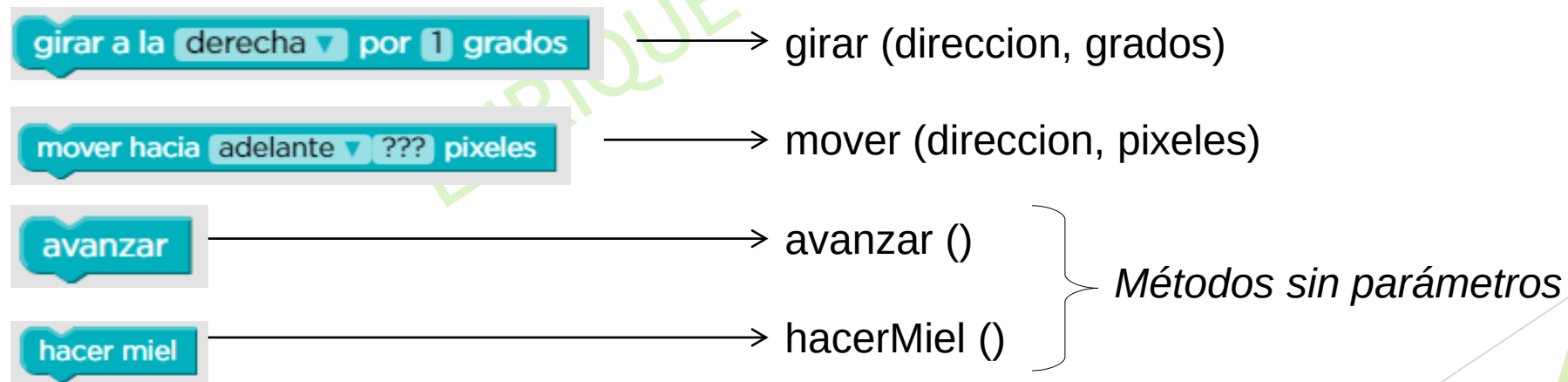
- Piensa las propiedades y comportamientos del objeto hombre (el muñeco del juego):
 - Propiedades: ¿?
 - Comportamientos: ¿?



UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

1 - Principios de la programación O.O.

- Cuando programemos en Java, llamaremos MÉTODOS a los comportamientos de un objeto.
- Los métodos se denotan con un nombre y una lista de parámetros entre paréntesis que especifican **la información que necesitan conocer para poder realizar su tarea.**



UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

1 - Principios de la programación O.O.

► «Todo objeto es de una clase»

- Los objetos se crean a partir de una clase.
- Una clase es el “molde” o modelo con el que se crean los objetos.
- La idea es crear la clase/molde y después podremos fabricar tantos objetos de ella como necesitemos.



UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

1 - Principios de la programación O.O.

- Una clase define la estructura y el comportamiento compartidos por los objetos que se puedan crear a partir de ella



Clase: PrototipoCoche



Objetos: coche1, coche2...



«A los objetos creados de una clase también se les llama **instancias**»

UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

1 - Principios de la programación O.O.

► «Los objetos se comunican entre ellos mediante mensajes»

- Los mensajes son peticiones que un objeto hace a otro para que realice alguna acción o comportamiento. Es decir, que un objeto puede usar los métodos de otro objeto.



Objeto Pedro de la clase Persona



arrancar()



parar()



Objeto Ibiza de la clase Coche

UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

1 - Principios de la programación O.O.

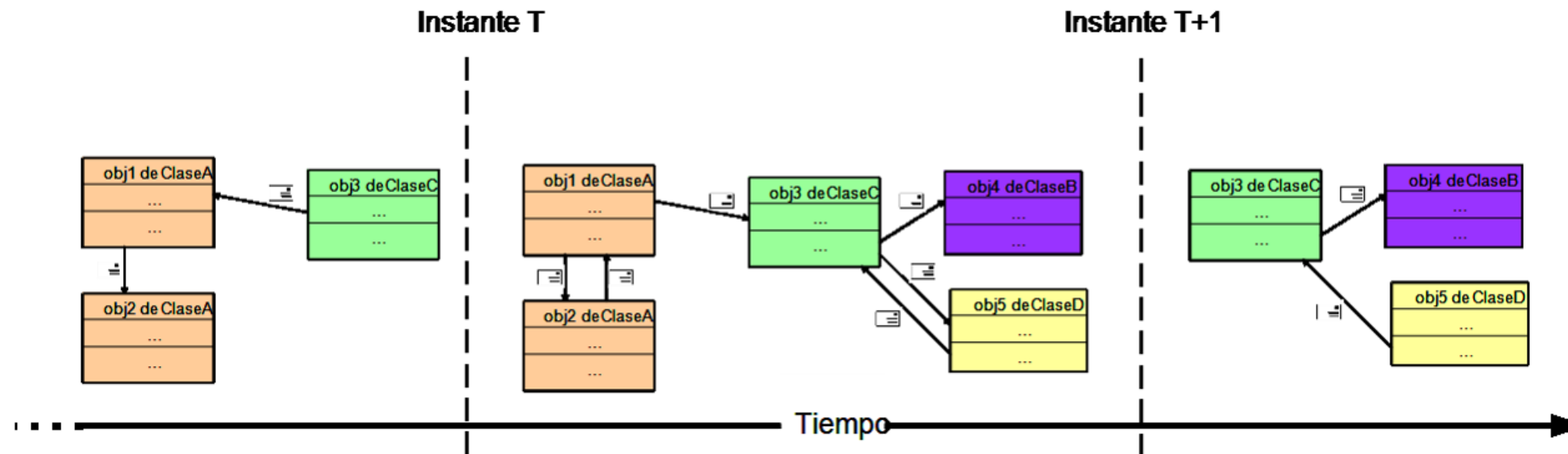
- ▶ «Una aplicación es una comunidad de objetos que interactúa para resolver un problema»
- ▶ Este principio sostiene que una aplicación es una comunidad de objetos que nacen (se crean a partir de una clase), interactúan entre ellos enviándose mensajes y mueren o desaparecen.
- ▶ Si se ha analizado correctamente el problema a resolver y se ha hecho un buen trabajo de programación de la interacción entre la comunidad de objetos se debe obtener la solución al problema



UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

1 - Principios de la programación O.O.

- En la siguiente figura se muestra una posible evolución de la comunidad de objetos que “habitan” en la memoria del ordenador

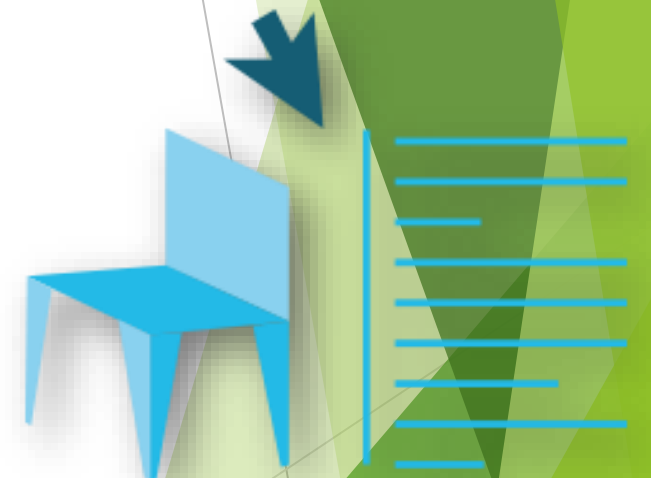


UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

1 - Principios de la programación O.O.

► 2.- Notación UML

- Existe una notación globalmente aceptada para representar a los objetos y las clases. Se trata de la notación UML (Unified Modeling Language)
- UML aporta un conjunto de diagramas que permiten describir una aplicación de forma parecida a como un arquitecto diseña un edificio (plano de forjado, plano de revestimiento, plano de instalación de agua...)
- Solo usaremos dos diagramas muy sencillos, el de clase y el de objeto-



UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

2 - Notación UML



Diagrama de clase

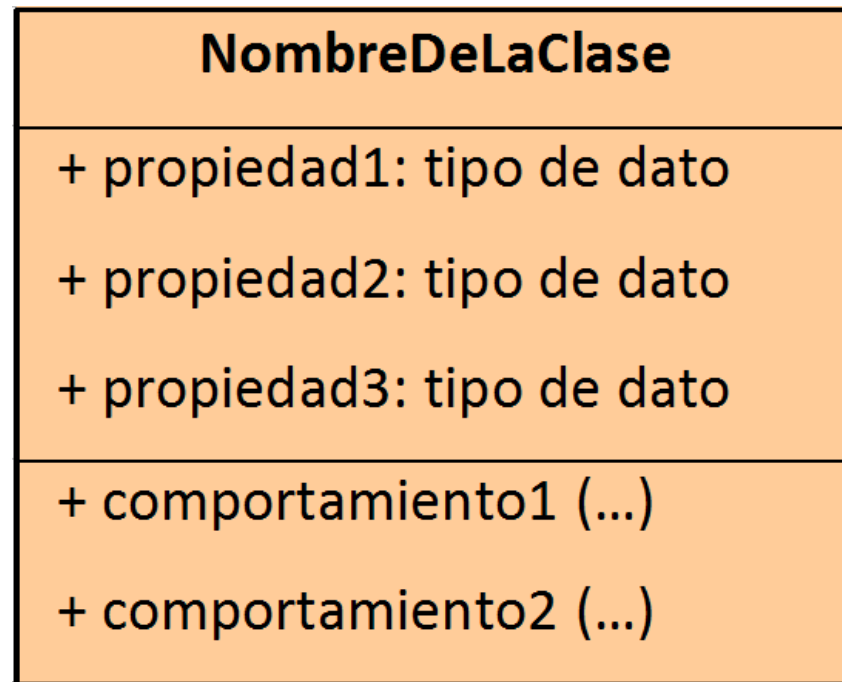
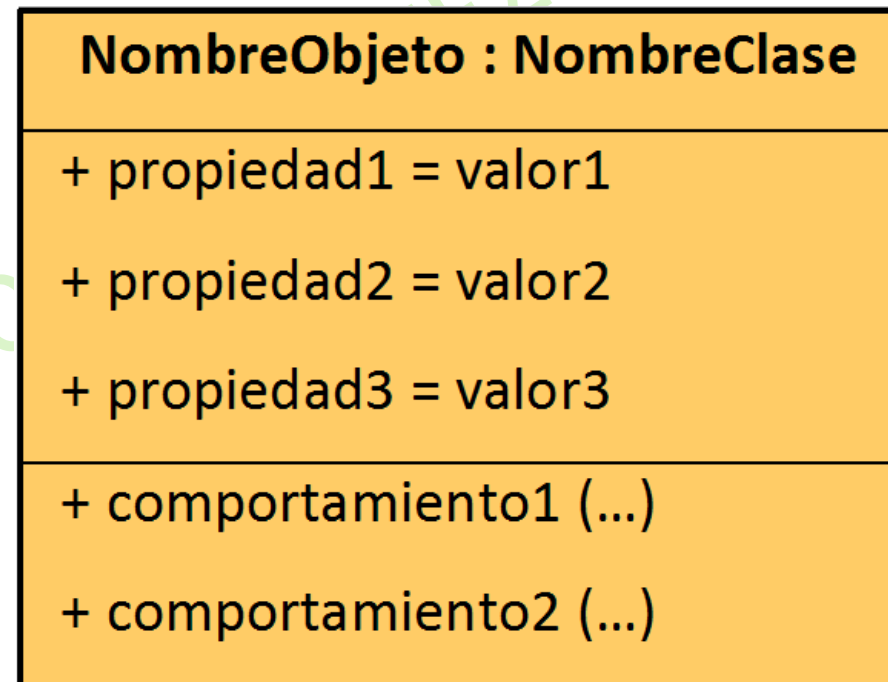


Diagrama de objeto



- ▶ «El símbolo + que precede a las propiedades y métodos indica que son de uso público. Por el momento, siempre usaremos este símbolo»

UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

2 - Notación UML

- Ejemplo de clase y objeto creado a partir de dicha clase

Clase Mechero
+longitud: real > 0 (en cm)
+peso: real > 0 (en gr)
+gasRestante: real $\geq$ 0 (en ml)
+encendido: lógico
+encender()
+apagar()
+rellenarGas (cantidad: real)

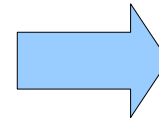
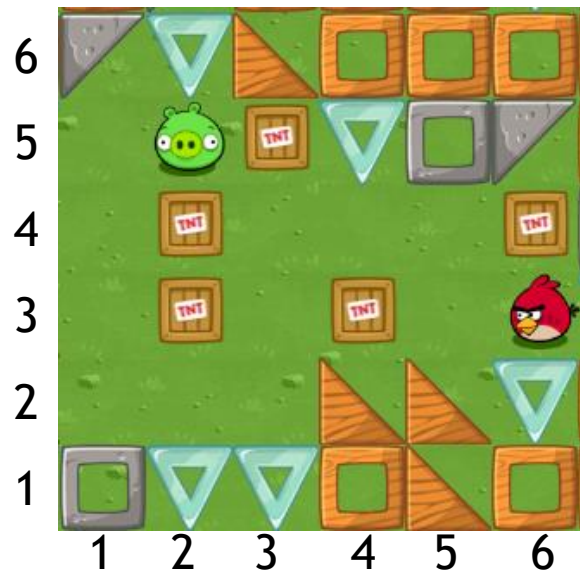
miMechero : Mechero
+longitud = 5.0
+peso = 10.3
+gasRestante = 3.4
+encendido = verdadero
+encender()
+apagar()
+rellenarGas (cantidad: real)

Recuerda que la clase es el MOLDE con el que fabricamos los objetos

UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

2 - Notación UML

► Ejemplos de objetos y su representación en UML



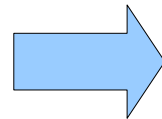
pajaroRojo : Pájaro
+ posiciónX = 6 + posiciónY = 3
+ avanzar () + girar (direccion)

*Una posible
representación
de un objeto de la clase
Pájaro*

UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

2 - Notación UML

► Otro ejemplo:



abeja1 : Abeja
+ posiciónX = 2 + posiciónY = 4
+ avanzar () + girar (direccion) + obtenerNéctar () + hacerMiel()

flor1 : Flor
+ posiciónX = 4 + posiciónY = 4 + cantidadNectar = 1
+ darNectar ()

flor2 : Flor
+ posiciónX = 2 + posiciónY = 2 + cantidadNectar = 1
+ darNectar ()

flor3 : Flor
+ posiciónX = 4 + posiciónY = 2 + cantidadNectar = 1
+ darNectar ()

Posibles representaciones de un objeto de la clase Abeja y tres de la clase Flor

UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

2 - Notación UML

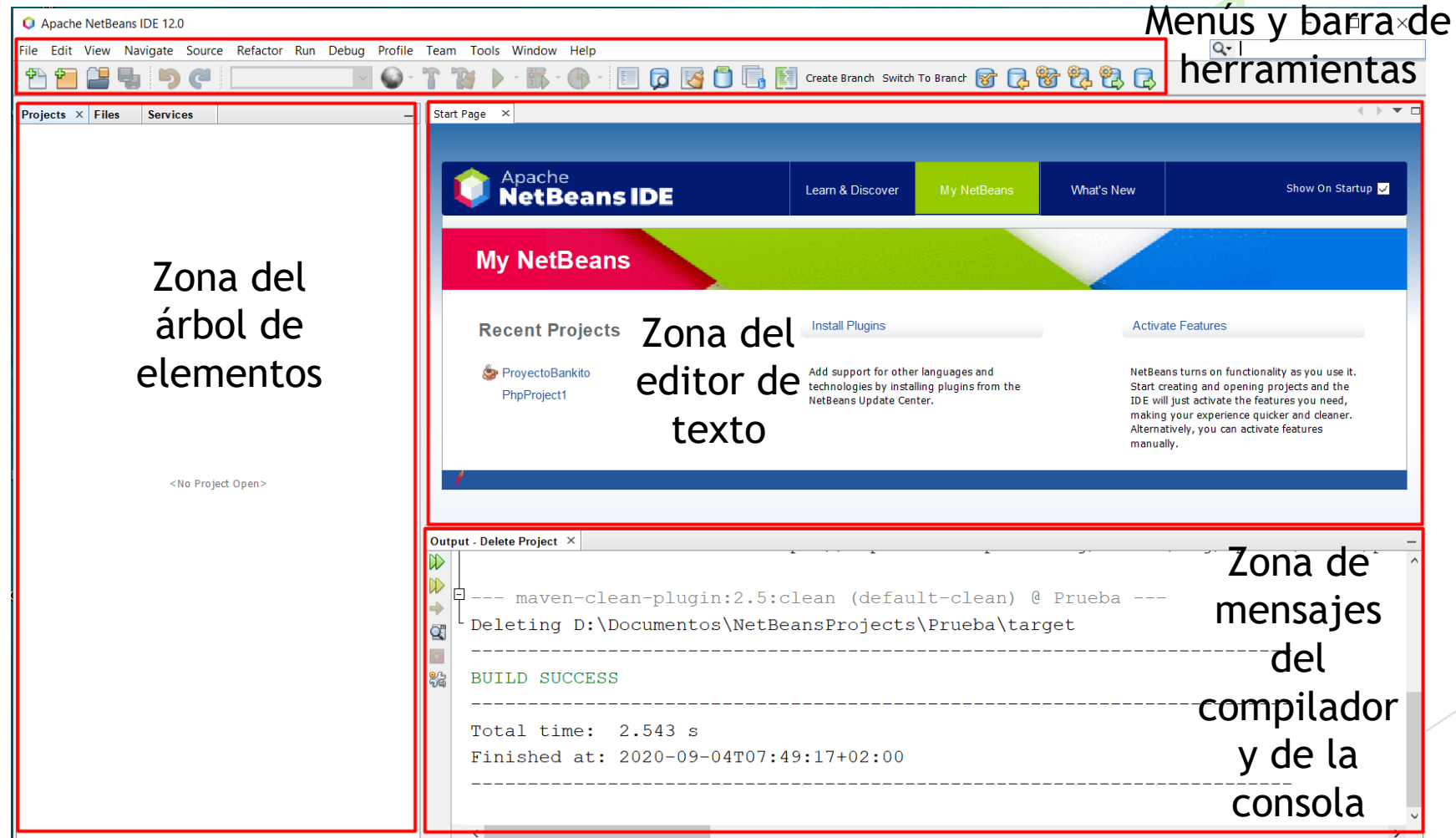
► Cuestiones:

- 1) Si un problema puede ser representado como un conjunto de objetos que interactúan entre ellos, *¿qué objetos identificas cuando quieres sacar dinero de un cajero automático?*
- 2) ¿Qué propiedades y comportamientos podría tener un objeto de la clase Coche?
- 3) Dibuja el diagrama UML de la clase Coche anterior y del objeto “fordFocus” creado a partir de la clase Coche.
- 4) Pedro es un objeto de la clase Persona que es dueño del objeto “fordFocus”, ¿qué mensajes tendría que enviar Pedro a su objeto para recorrer 100 km?

UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

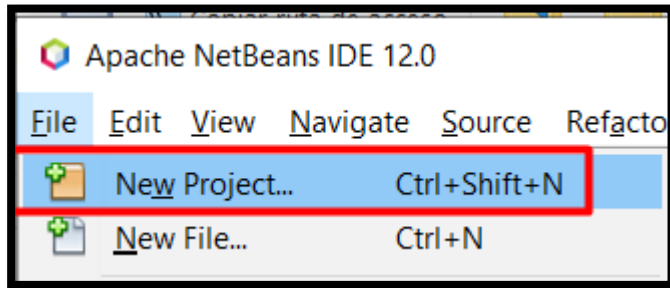
3 - Primer programa con NetBeans: HolaMundo

► 3 – Primer programa con NetBeans: HolaMundo

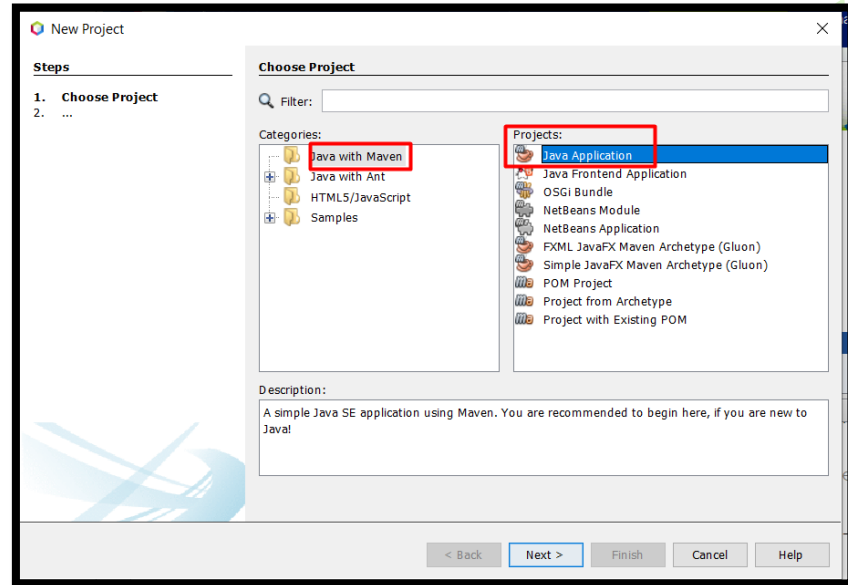


UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

3 - Primer programa con NetBeans: HolaMundo



Creamos un proyecto de software

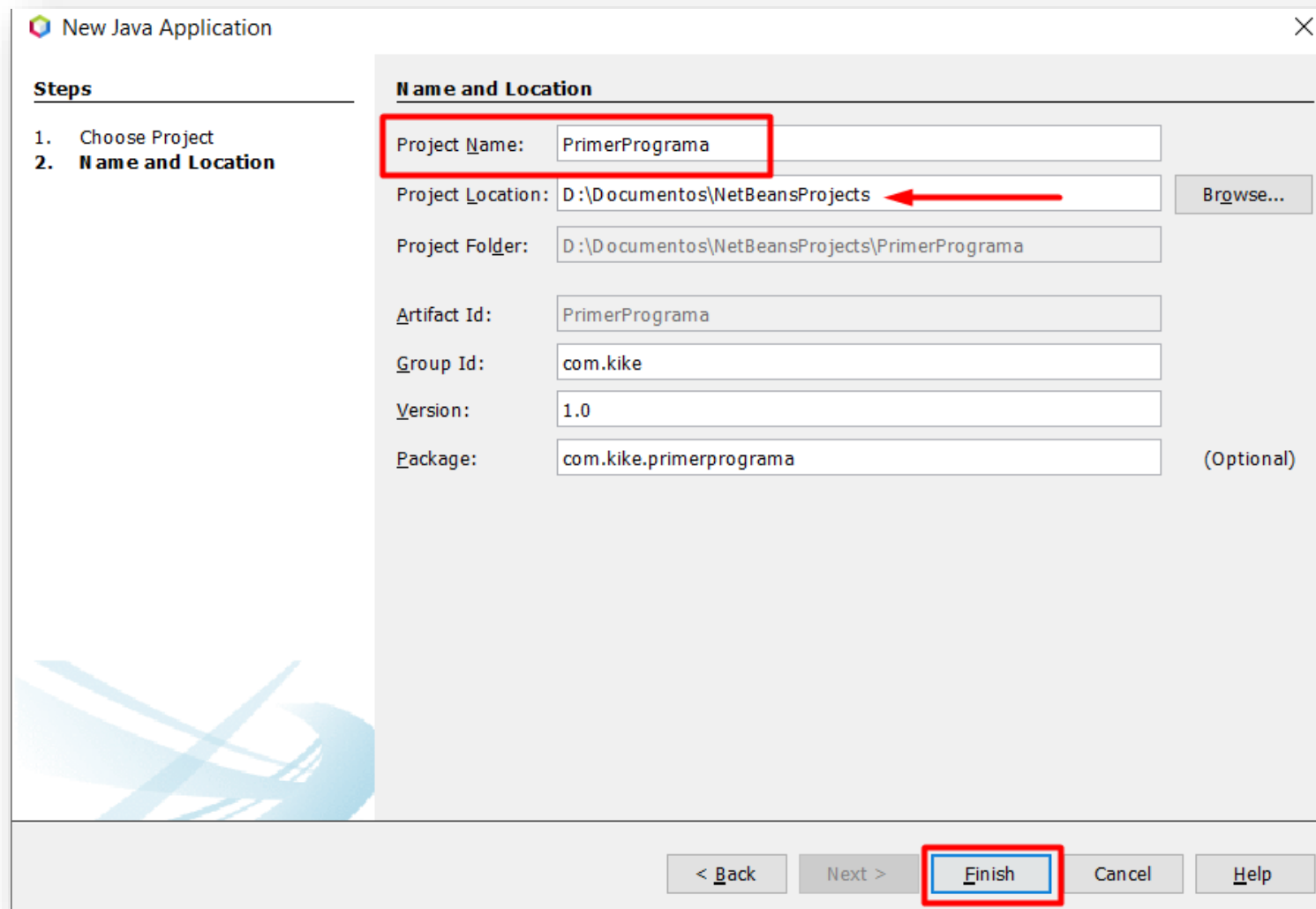


Le indicamos que queremos una Java Application dentro de la categoría Java with Maven

Maven es un gestor de dependencias de librerías, lo estudiaréis con detenimiento en el módulo de Entornos de Desarrollo

UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

3 - Primer programa con NetBeans: HolaMundo



New Java Application

Steps

1. Choose Project
2. **Name and Location**

Name and Location

Project Name: PrimerPrograma

Project Location: D:\Documentos\NetBeansProjects Browse...

Project Folder: D:\Documentos\NetBeansProjects\PrimerPrograma

Artifact Id: PrimerPrograma

Group Id: com.kike

Version: 1.0

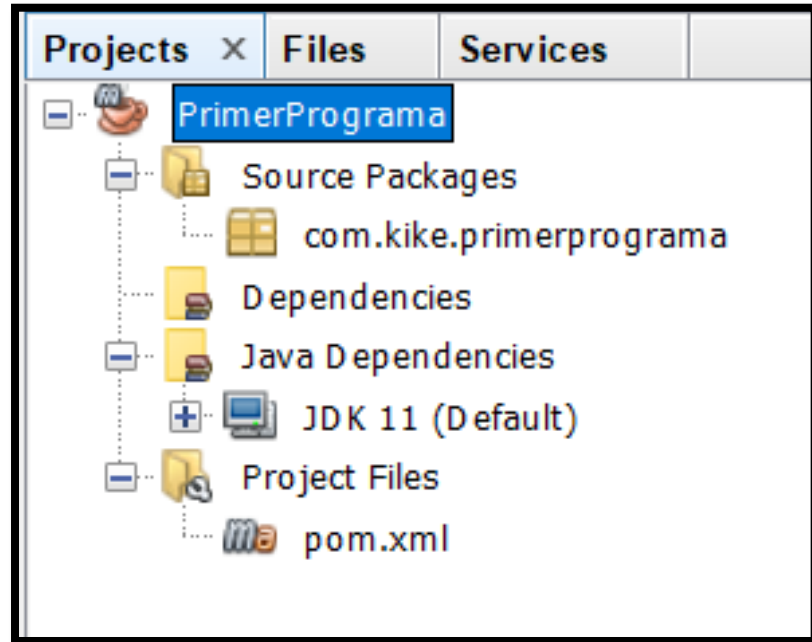
Package: com.kike.primerprograma (Optional)

< Back Next > Finish Cancel Help

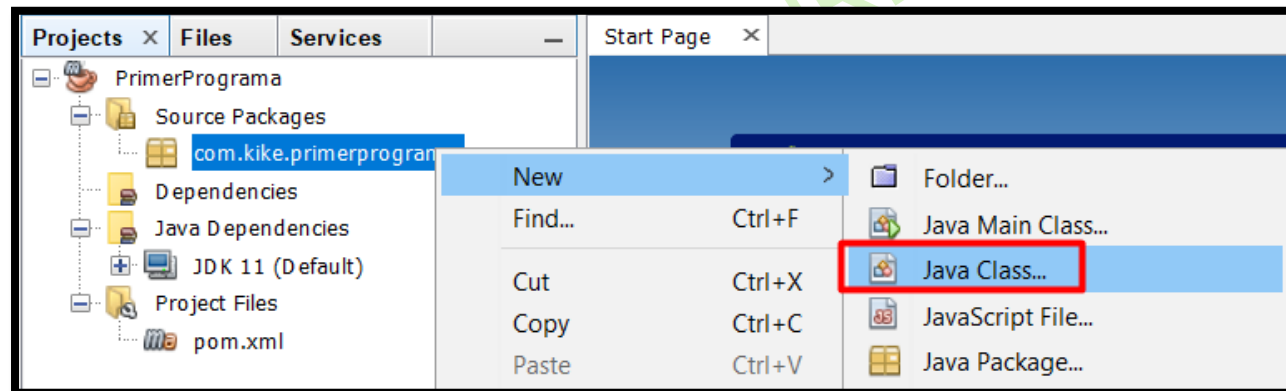
Indicamos el nombre del proyecto y observamos la ruta donde se va a guardar

UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

3 - Primer programa con NetBeans: HolaMundo



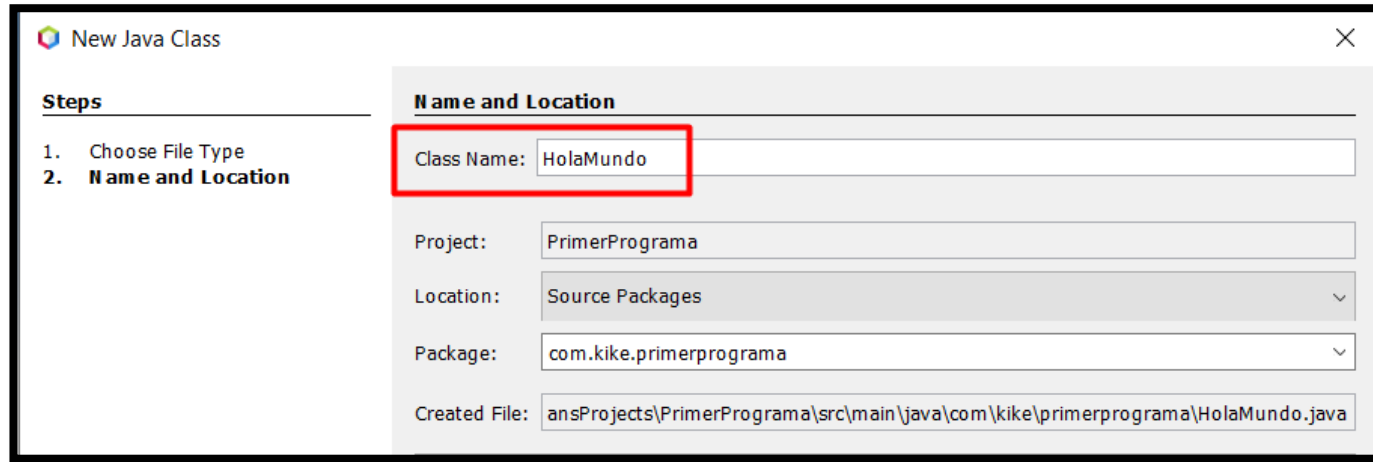
Si pinchamos en + para desplegar vemos que no hay mucho en el árbol de elementos



Hacemos click con el botón derecho del elemento que cuelga de Source Packages y seleccionamos New -> Java Class

UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

3 - Primer programa con NetBeans: HolaMundo



Ponemos HolaMundo (ojo al escribirlo) y pinchamos en finalizar

```
1  /*
2   * To change this license header, choose License Headers in Project Properties.
3   * To change this template file, choose Tools | Templates
4   * and open the template in the editor.
5   */
6  package com.kike.primerprograma;
7
8  /**
9   *
10  * @author Kike
11  */
12  public class HolaMundo {
13
14  }
```

Se abrirá un fichero llamado HolaMundo.java con el “esqueleto” de nuestro programa

UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

3 - Primer programa con NetBeans: HolaMundo

```
public class HolaMundo {  
    public static void main (String args[]) {  
        System.out.println("Hola Mundo!!");  
    }  
}
```

Ahora vamos a escribir el siguiente código tal cual, no puede faltar ni sobrar nada

Para desplazar el código a la derecha se utiliza la tecla TAB una o más veces



Esta forma de presentar el código con distintos niveles de “sangría” o “indentado” nos facilitará una buena lectura y comprensión del código. Lo trabajaremos durante todo el curso

UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

3 - Primer programa con NetBeans: HolaMundo

```
12 public class HolaMundo {  
13  
14     public static void main (String args[]) {  
15         Sistem.out.println("Hola Mundo!!");  
16     }  
17  
18 }
```

Si nos equivocamos en algo NetBeans lo marca en rojo

```
12 public class Ho  
13  
14     package Sistem does not exist  
15     ----  
16     (Alt-Enter shows hints)  
17  
18     Sistem.
```

Y si ponemos el cursor sobre la bombillita nos dará una pista del motivo del error

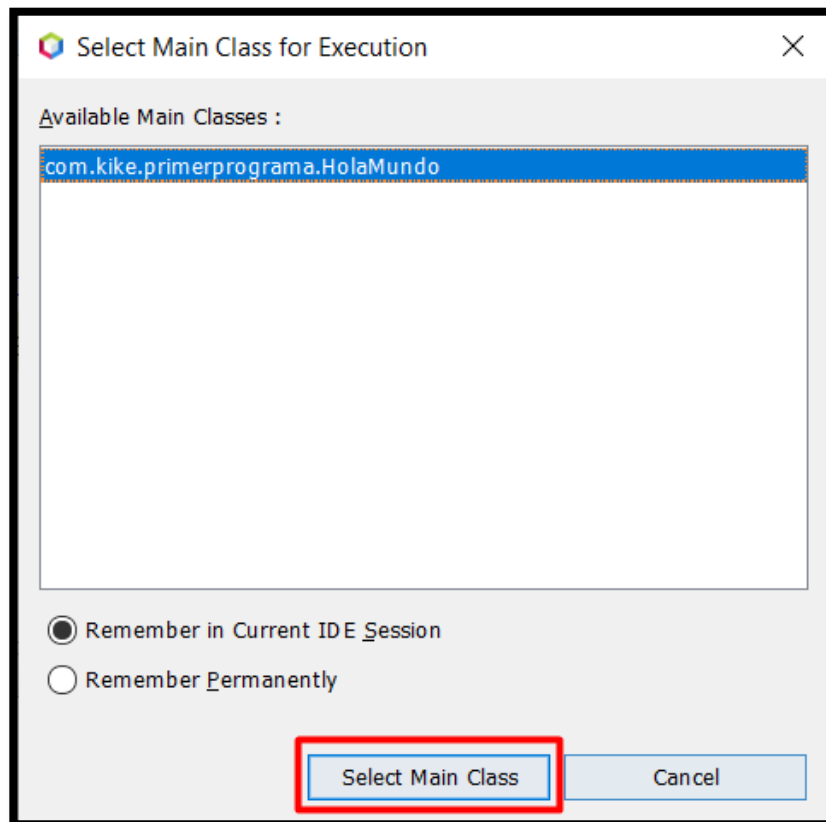
UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

3 - Primer programa con NetBeans: HolaMundo

Si no tenemos errores, salvamos nuestro archivo con el icono:



Y por último, compilamos y ejecutamos con el botón:

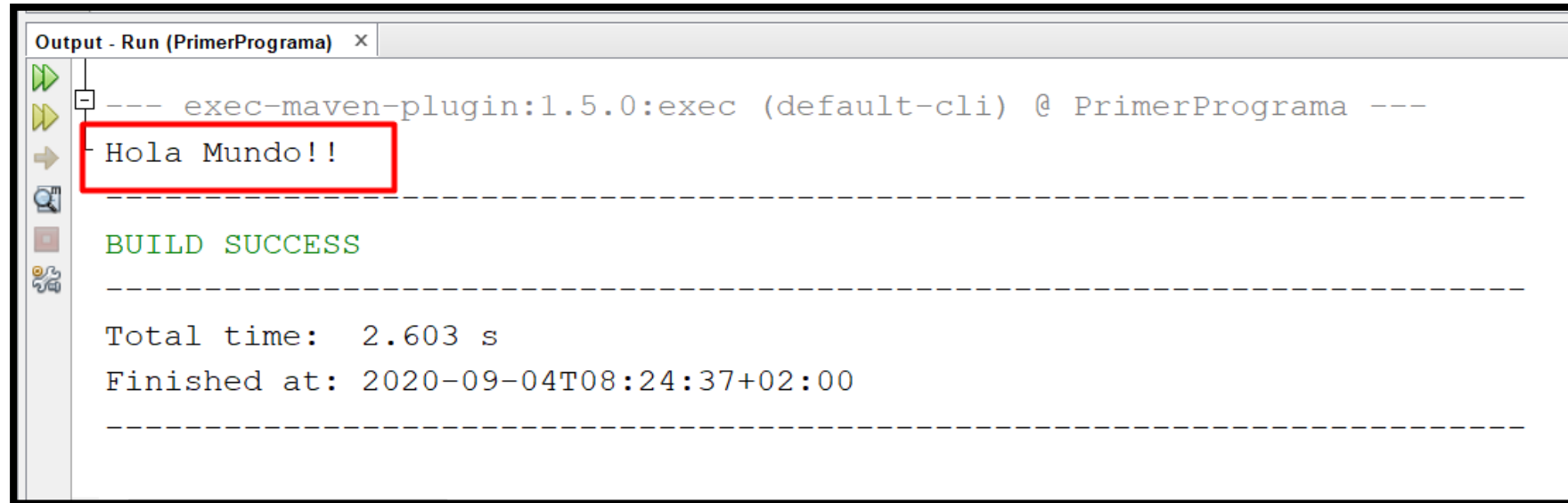


A continuación, NetBeans nos pregunta cuál es la “clase principal”, que es la clase por donde empezará a ejecutarse nuestro programa. Como solo tenemos un fichero, lo seleccionamos

*La clase principal, en este momento del curso, es aquella que contiene la línea:
`public static void main (String args[])`*

UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

3 - Primer programa con NetBeans: HolaMundo



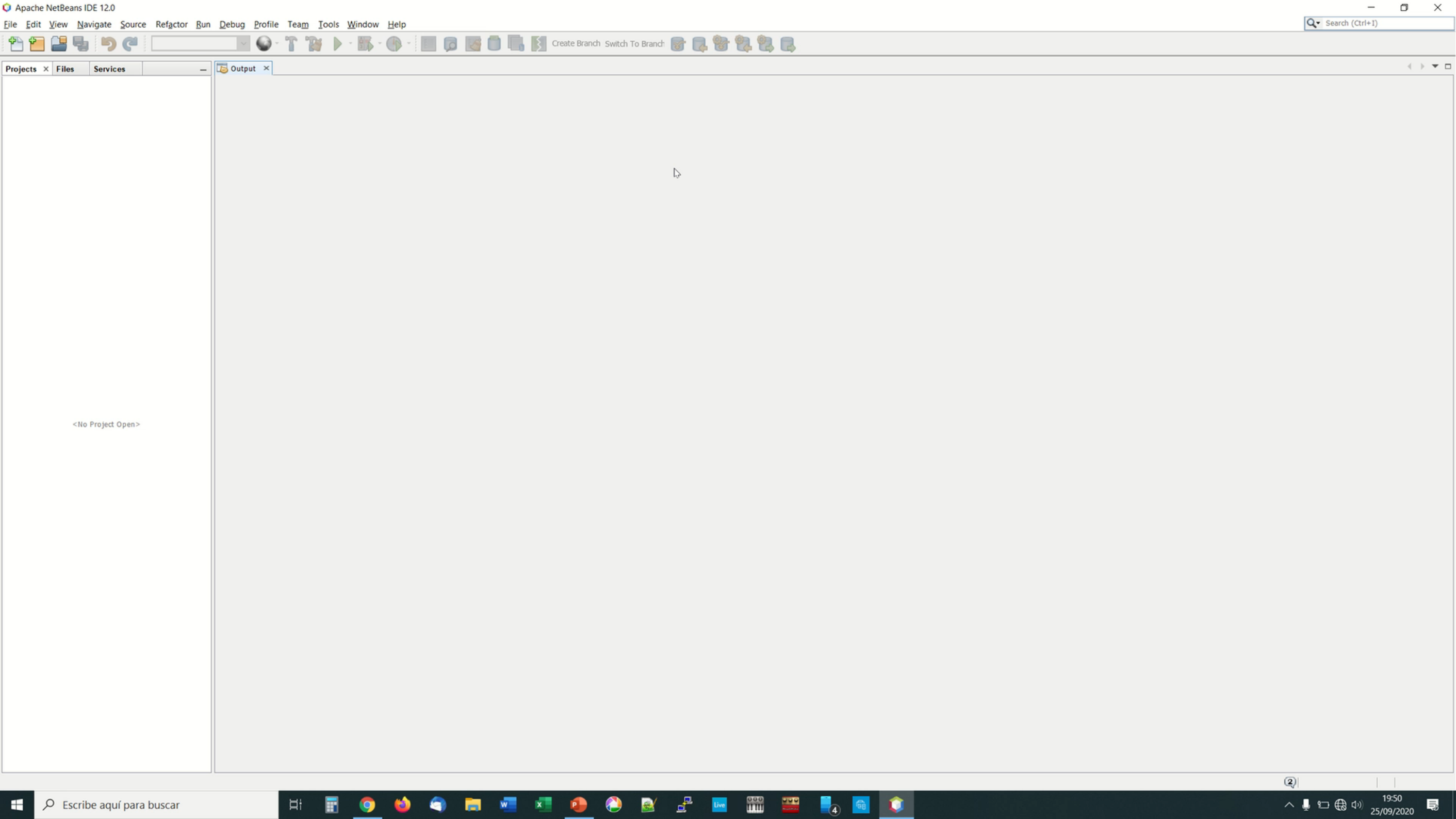
```
Output - Run (PrimerPrograma) x
--- exec-maven-plugin:1.5.0:exec (default-cli) @ PrimerPrograma ---
Hola Mundo!!
BUILD SUCCESS
Total time: 2.603 s
Finished at: 2020-09-04T08:24:37+02:00
```

En la ventana de salida (output) observaremos que nuestro programa ha escrito el mensaje “Hola Mundo!!”.

Modifica el código y repetir el proceso para que escriba en la pantalla “Hola *tu nombre*” (Hola Pepe, Hola María...)

漢字 汉字

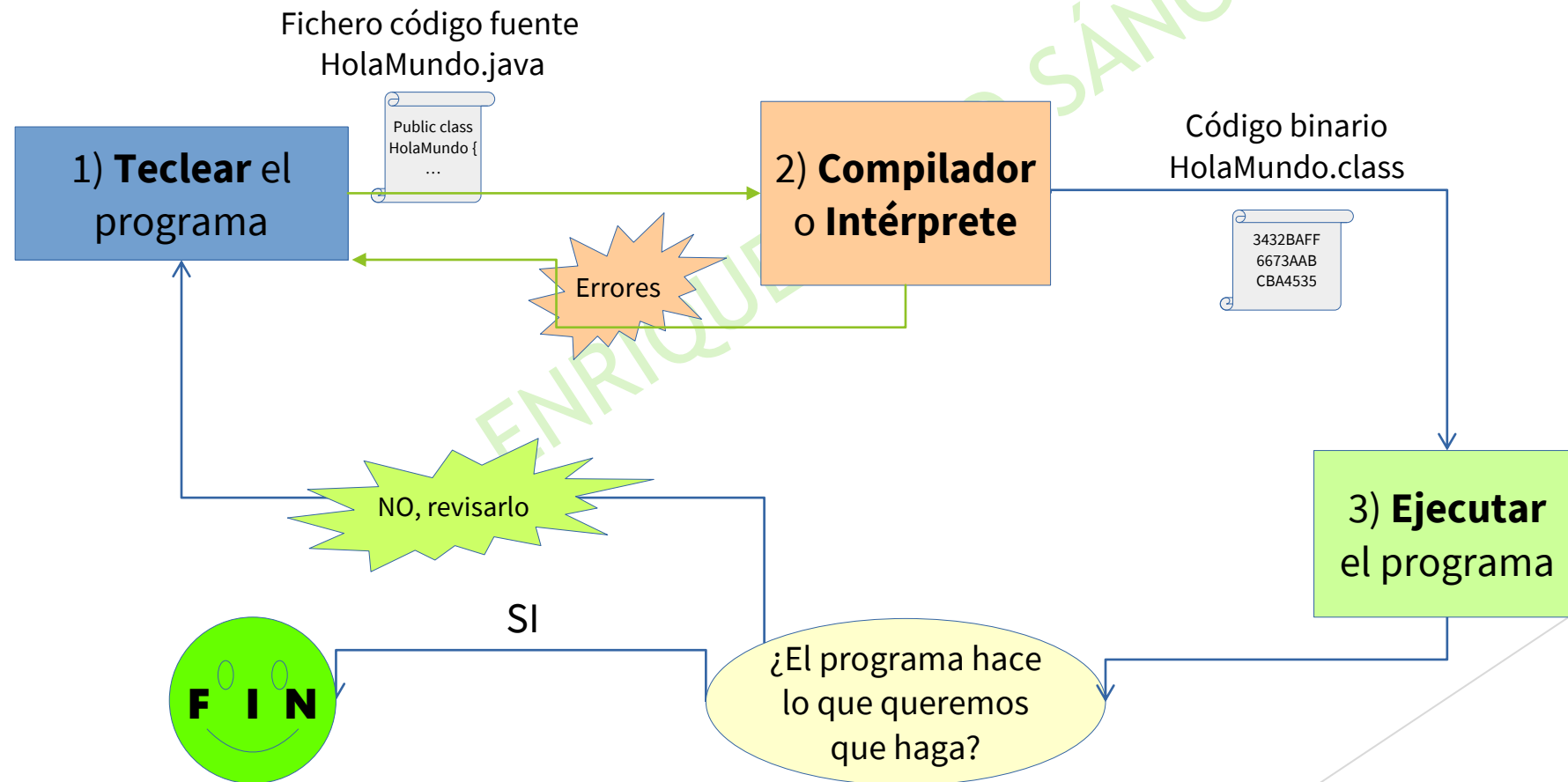
*Es normal que todo esto
te suene a CHINO.
Hay que tener paciencia,
estamos aprendiendo un
“idioma nuevo” y el
cerebro se toma su
tiempo.*



UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

3 - Primer programa con NetBeans: HolaMundo

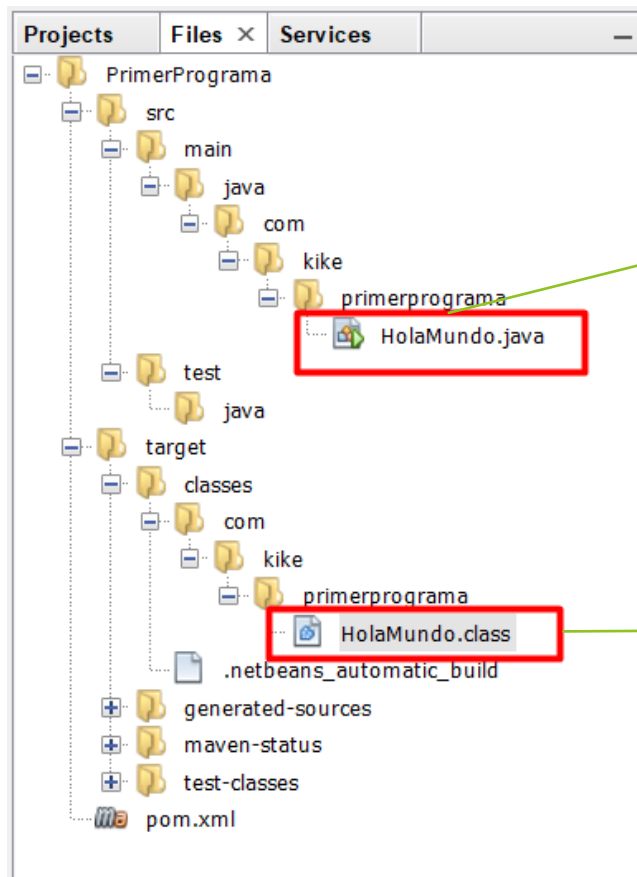
Si ahora volvemos a mirar el diagrama del tema anterior en el que se explican las fases típicas de la programación, ahora tiene más sentido.



UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

3 - Primer programa con NetBeans: HolaMundo

Por último, si queremos ver los ficheros implicados en el proceso, podemos ir a la pestaña Files y ver tanto el fichero de código fuente como el fichero de código binario que ejecuta el ordenador:



```
package com.kike.primerprograma;

public class HolaMundo {

    public static void main (String args[]) {
        System.out.println("Hola Mundo!!");
    }

}
```

```
? °¾ 7 #
STX ETXBEL EOTFF ENO ACKSOH DLEjava/lang/ObjectSOH AC
FF VT FFSOH DLEjava/lang/SystemSOH ETXoutSOH NAKLjava/
DLE DC1BEL DC2FF DC3 DC4SOH DC3java/io/PrintStreamSOH
SourceFileSOH SOHolaMundo.java ! NAK STX STX SOH EN
¶ SI± STX CAN
STX SI BS DLE EM FF SOH RS US ENOSOH
```

UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

4 - Elementos básicos del lenguaje

4.- Elementos básicos del lenguaje

Aprender un lenguaje de programación es como aprender un idioma, requiere disciplina y constancia... No puedes aprobar un B2 de inglés estudiando la tarde antes del examen si no has hecho nada durante meses.

Seguiremos un **APRENDIZAJE EN ESPIRAL**:

- Al principio aprendemos unas pocas palabras y las reglas básicas para hacer un frases y seremos capaces de hacer frases sencillas.
- Más adelante aprendemos más palabras y nuevas reglas para hacer frases y nuestro discurso se va haciendo más rico...



Para aprender un lenguaje o un idioma el camino más efectivo no es la línea recta si no la espiral.

UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

4 - Elementos básicos del lenguaje

4.1- Primera vuelta a la espiral

En esta vuelta sentaremos las bases para escribir un programa sencillo y para ello vamos a estudiar los siguientes elementos:

Comentarios

Los comentarios nos ayudan a clarificar el código, anotar ideas y documentar nuestro programa. Hay dos tipos de comentarios:

// Comentario breve de 1 sola línea

/* Esto es un comentario de varias líneas. Se abre con una barra y un asterisco y se cierra con un asterisco y una barra. */

«Los comentarios son IGNORADOS por el compilador. Nunca serán ejecutados por el procesador del ordenador. Solo sirven para aclararnos las ideas»

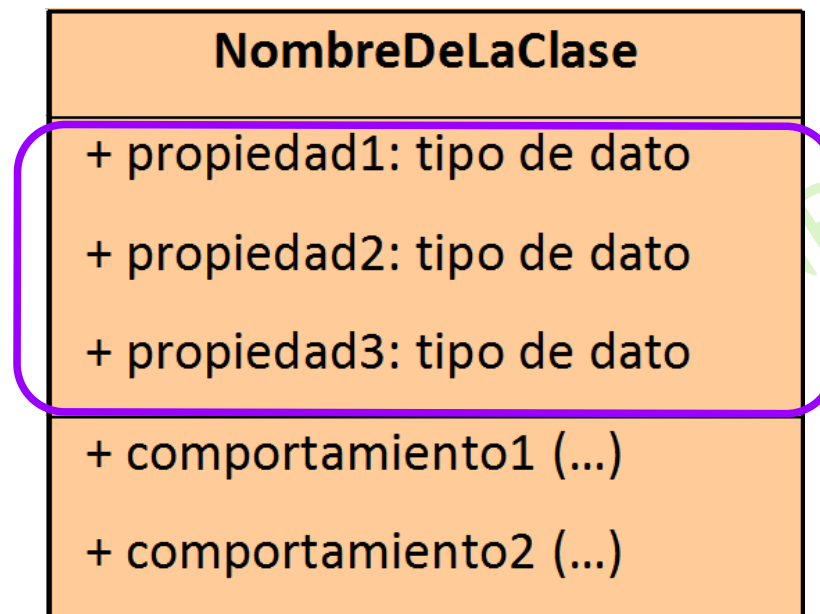


UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

4 - Elementos básicos del lenguaje

Clases

Recordemos que las clases son el “molde” o “plantilla” con la que creamos los objetos. Veamos cómo convertir el diagrama UML en un código Java equivalente. Observemos en las **propiedades**:



*Fichero **NombreDeLaClase.java***

```
public class NombreDeLaClase {  
    // Propiedades...  
    // Métodos o comportamientos  
}
```

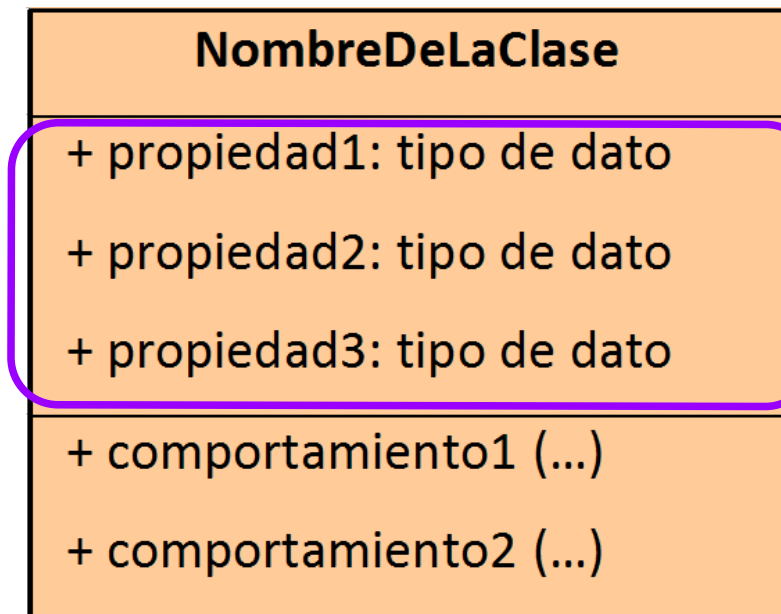
*ESTILO DE NOMBRADO:
EstiloDeNombreDeUnaClase*

UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

4 - Elementos básicos del lenguaje

Propiedades

Son los atributos que definen las características de una clase.



Fichero *NombreDeLaClase.java*

```
public class NombreDeLaClase {  
    // Propiedades  
    tipoDeDato propiedad1;  
    tipoDeDato propiedad2;  
    tipoDeDato propiedad3;  
    // Métodos o comportamientos  
}
```

A red question mark icon is placed to the left of the property declarations. A red rounded rectangle highlights the words "tipoDeDato" in the first three lines. An orange rectangle highlights the semicolons at the end of the first three lines. A green diagonal watermark "RODRIGO SÁNCHEZ" is visible across the code block.

ESTILO DE NOMBRADO:
estiloDeNombreDeUnaPropiedad

UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

4 - Elementos básicos del lenguaje

► *Los tipos de datos*

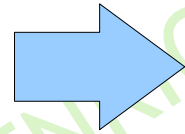
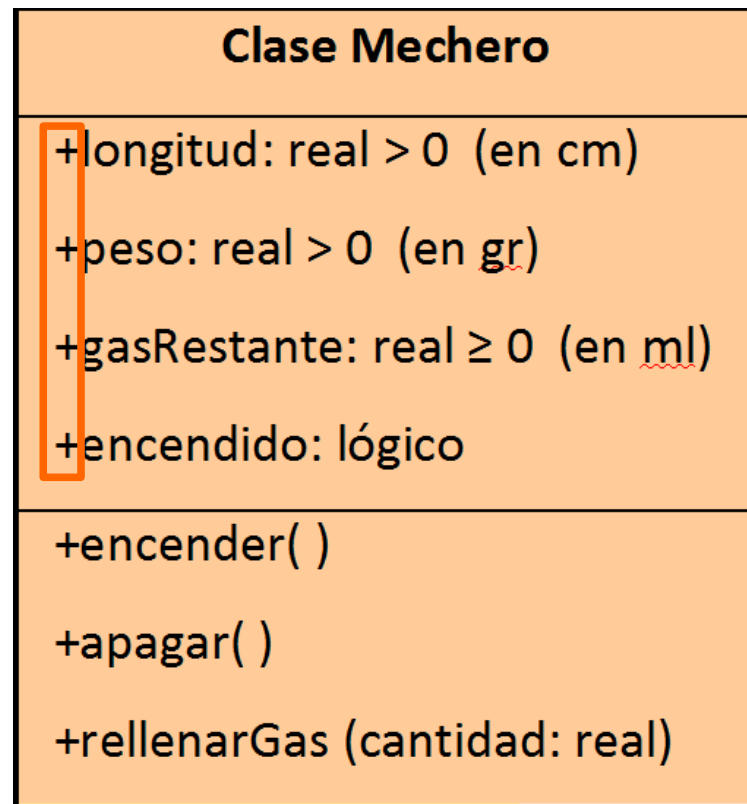
TIPOS DE DATOS	CLASIFICACIÓN	NOMBRE	TIPO	OCUPA EN MEMORIA	RANGO APROXIMADO
	TIPOS PRIMITIVOS (elementos simples como números o caracteres)	byte	Número entero	1 byte	-128 a 127
		<u>short</u>	Número entero	2 bytes	-32.768 a 32.767
		<u>int</u>	Número entero	4 bytes	-2x10 ⁹ a 2x10 ⁹
		<u>long</u>	Número entero	8 bytes	Muy grande
		<u>float</u>	Número real (precisión normal)	4 bytes	Muy grande
		<u>double</u>	Número real (precisión doble)	8 bytes	Muy grande
		<u>char</u>	Carácter simple	2 bytes	---
		<u>boolean</u>	Valor true o false	1 byte	---
	TIPOS OBJETO (con métodos, necesitan una invocación para ser creados)	Tipos de la biblioteca estándar de Java	<u>String</u> (cadenas de texto)...		
		Tipos definidos por el programador/a	Cualquier clase que programemos: Mechero, Bombilla, Coche, <u>TarjetaCredito</u> ...		

Marcamos en **negrita** los más habituales

UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

4 - Elementos básicos del lenguaje

- Conociendo los tipos de datos ya podemos definir las propiedades de una clase:



Fichero **Mechero.java**

```
public class Mechero {  
    // Propiedades  
    public double longitud;  
    public double peso;  
    public double gasRestante;  
    public boolean encendido;  
  
    // Métodos o comportamientos  
    // ...  
}
```



Se tienen que llamar exactamente igual

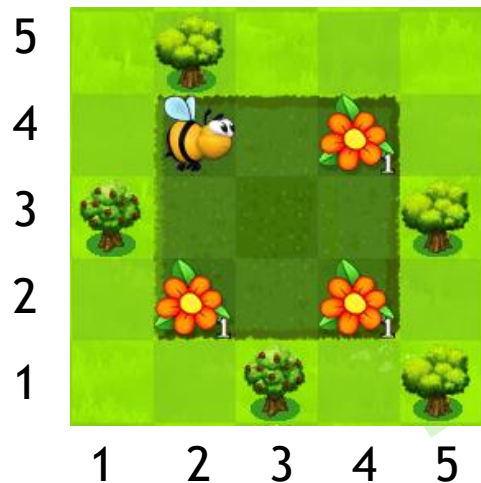
“public” indica un nivel de permiso de acceso para el resto de los objetos del programa. Si una propiedad es “pública” entonces cualquier objeto podrá consultar y modificar los valores de nuestras propiedades

UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

4 - Elementos básicos del lenguaje

► Ejercicio grupal 1

Vamos a crear un nuevo proyecto llamado U1.P1.Abeja. Tecleamos y compilamos el código correspondiente a la siguiente clase.



Abeja
+ posicionX: entero
+ posicionY: entero
+ numCargasNectar: entero
+ volando: lógico
+ peso: real

*Si no tenemos errores de sintaxis el compilador nos debe devolver el mensaje: **'BUILD SUCCESS'***



UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

4 - Elementos básicos del lenguaje

► Ejercicio grupal 2

Creamos un nuevo proyecto llamado U1.P2.Persona. Tecleamos y compilamos el código correspondiente a la siguiente clase.



Persona
+ nombre: cadena texto + apellido1: cadena texto + apellido2: cadena texto + dni: texto + peso: real + soltero: lógico

*Si no tenemos errores de sintaxis el compilador nos debe devolver el mensaje: **'BUILD SUCCESS'***



UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

4 - Elementos básicos del lenguaje

► **Métodos**

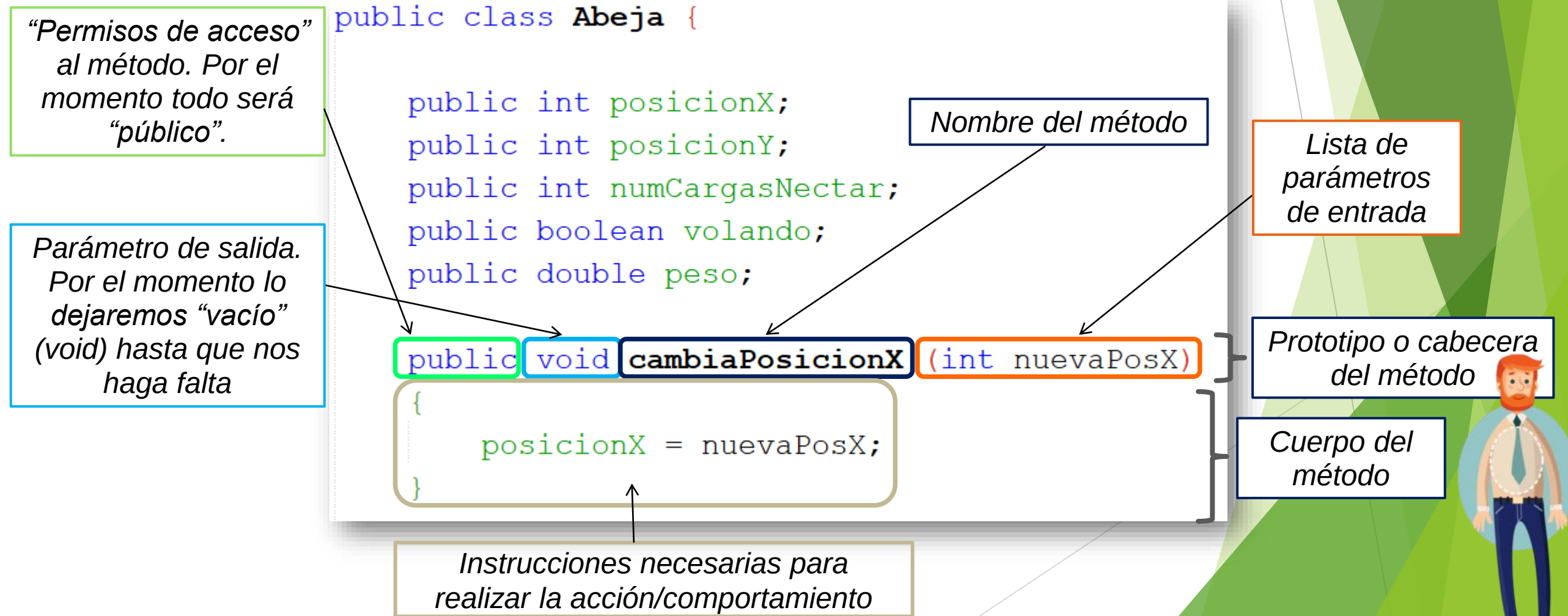
- Los métodos son las acciones o comportamientos que puede realizar un objeto fabricado a partir de una clase.
- Siempre van acompañados de una lista de parámetros entre paréntesis. Aunque esta lista puede estar vacía.
- Estos parámetros representan a los datos que el método necesita conocer para poder realizar su tarea.
- Empezaremos con unos métodos sencillos que cambian o imprimen el valor de las propiedades.

Abeja
+ posicionX: entero + posicionY: entero + numCargasNectar: entero + volando: lógico + peso: real
+ cambiaPosicionX (nuevaPosX: entero) + cambiaPosicionY (nuevaPosY: entero) + cambiaCargaNectar (nuevaCarga: entero) + cambiaPeso (nuevoPeso: real) + imprimePosicionX () + imprimePosicionY () + imprimeNumCargasNectar() + imprimePeso() + vuela() + sePosa() + imprimeEstadoVuelo()

UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

4 - Primeras palabras del lenguaje

- Analicemos la estructura básica de un método:



UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

4 - Primeras palabras del lenguaje

► Observemos cada elemento:

- **Permiso de acceso:** permite gestionar el acceso que tienen otros objetos sobre este método. “public” significa que cualquier objeto podría utilizar este método.
- **Nombre del método:** debe ser descriptivo del comportamiento o acción que representa. Normalmente se usan verbos en infinitivo: mostrarPosicion, volar, aterrizar... Se sigue el estilo de nombrado siguiente: ***nombreDelMetodo*** (ojo que la primera letra va en minúsculas)
- **Lista de parámetros de entrada:** cada parámetro representa un dato que necesita conocer el método para poder realizar su acción. Cada parámetro lleva un nombre y su tipo de datos. Siempre se escriben entre paréntesis.



UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

4 - Primeras palabras del lenguaje

- **Lista de instrucciones:** para realizar la acción asociada al método el ordenador deberá ejecutar una lista de instrucciones. Estas se escriben encerradas entre dos llaves { } que indican el comienzo y fin de la lista de instrucciones.

- La instrucción que se está ejecutando se le llama “**asignación**” e indica que el valor de la parte derecha se le **asigna/copia** a la parte izquierda. Nunca debe entenderse como una igualdad matemática.

- En nuestro ejemplo lo que se está haciendo es que el valor que entra como parámetro de entrada se copia sobre la propiedad *posicionX*.

```
public class Abeja {  
  
    public int posicionX;  
    public int posicionY;  
    public int numCargasNectar;  
    public boolean volando;  
    public double peso;  
  
    public void cambiaPosicionX (int nuevaPosX)  
    {  
        posicionX = nuevaPosX;  
    }  
}
```



UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

4 - Primeras palabras del lenguaje

- **Parámetro de salida:** en un futuro próximo un método aprenderemos que un método puede informar de un dato que haya calculado en su lista de instrucciones, por ejemplo, la suma de dos números o la nota media de un alumno/a. Esto se correspondería con un parámetro de salida, aunque por el momento lo dejaremos a “vacío” (void en inglés).

漢字 汉字

*¡UFF! Volvemos al chino. Toca tener un poco de paciencia...
Poco a poco las piezas irán encajando y todo tendrá sentido*

UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

4 - Elementos básicos del lenguaje

- Veamos una variante más antes de hacer un ejercicio:

Abeja.java x

```
1 public class Abeja {
2
3     public int posicionX;
4     public int posicionY;
5     public int numCargasNectar;
6     public boolean volando;
7     public double peso;
8
9     public void cambiaPosicionX (int nuevaPosX) {
10         posicionX = nuevaPosX;
11     }
12
13     public void imprimePosicionX () {
14         System.out.println("Posicion X: " + posicionX);
15     }
16 }
```

No necesita parámetros

Imprime en pantalla el texto entre "" y a continuación el valor de posicionX

OJO: la S es mayúscula, si no da error

UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

4 - Elementos básicos del lenguaje

- ▶ Por último, estudiamos, debatimos y compilamos el código de la clase Abeja que está en la plataforma.
- ▶ Siempre pregunta tus dudas. No te cortes...

```
Abeja.java x
2
3 public int posicionX;
4 public int posicionY;
5 public int numCargasNectar;
6 public boolean volando;
7 public double peso;
8
9 public void cambiaPosicionX (int nuevaPosX) {
10     posicionX = nuevaPosX;
11 }
12
13 public void imprimePosicionX () {
14     System.out.println("Posicion X: " + posicionX);
15 }
16
17 public void cambiaPosicionY (int nuevaPosY) {
18     posicionY = nuevaPosY;
19 }
20
21 public void imprimePosicionY () {
22     System.out.println("Posicion Y: " + posicionY);
23 }
24
25 public void cambiaCargaNectar (int nuevaCarga) {
26     numCargasNectar = nuevaCarga;
27 }
28
29 public void imprimeNumCargasNectar () {
30     System.out.println("Número de cargas de nectar: " + numCargasNectar);
31 }
32
33 public void cambiaPeso (double nuevoPeso) {
34     peso = nuevoPeso;
35 }
36
```

UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

4 - Elementos básicos del lenguaje

► Ejercicio grupal 3

Crea un nuevo proyecto llamado U1.P3.PersonaMetodos e intenta codificar la siguiente clase:



Persona
+ nombre: cadena texto + apellidos: cadena texto + edad: entero
+ cambiaNombre(nombreNuevo: cadena texto) + cambiaApellidos (apellidosNuevo: cadena texto) + cambiaEdad (edadNueva: entero) + imprimeNombre() + imprimeApellidos() + imprimeEdad()

UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

4 - Elementos básicos del lenguaje

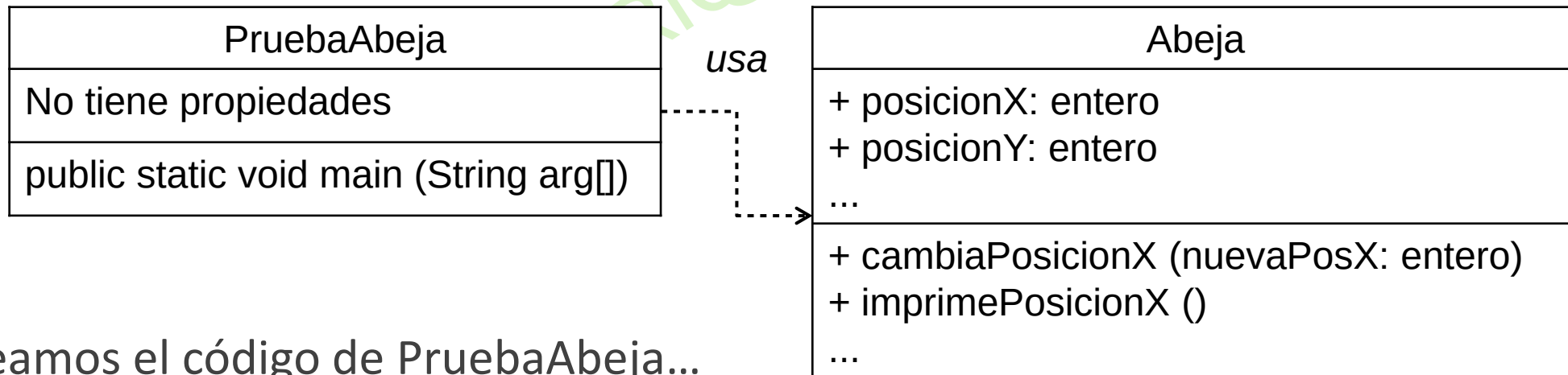
- ▶ Hasta ahora hemos aprendido a crear **CLASES** que son los **moldes** con los que podemos crear los objetos.
- ▶ Para terminar la primera vuelta de la espiral solo necesitamos aprender a **crear objetos a partir de una clase y utilizarlos**.
- ▶ Esto es fundamental ya que recordemos que uno de los principios de la programación orientada a objetos es: *“una aplicación es una comunidad de objetos que interactúa para resolver un problema”* así que nuestros programas tendrán muchas clases que nos permitirán fabricar muchos objetos distintos.



UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

4 - Elementos básicos del lenguaje

- ▶ De entre tantos objetos y clases tendremos que identificar algún punto del código como “**punto de comienzo**”, es decir, el que lanza la ejecución de nuestro programa de la forma correcta. Es como el primer capítulo de un libro.
- ▶ Para ver estos conceptos nuevos tomaremos como ejemplo la clase Abeja y crearemos otra clase llamada PruebaAbeja que será nuestro “**punto de comienzo**” y que creará un objeto de la clase Abeja y lo utilizará. En UML tendríamos.



- ▶ Veamos el código de PruebaAbeja...



UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

4 - Elementos básicos del lenguaje

Solo tiene un método, un poco especial, que será EL PUNTO DE COMIENZO de nuestro programa

Definimos y creamos un objeto a partir de la clase

Utilizamos los métodos del objeto con la sintaxis: objeto.metodo(...)

PruebaAbeja.java x

```
1 public class PruebaAbeja {
2
3     public static void main (String arg[]) {
4
5         // Definimos una referencia capaz de apuntar a objetos de la clase Abeja
6         Abeja maya;
7
8         // Ahora CREAMOS UN NUEVO OBJETO de la clase Abeja y se lo
9         // asignamos a maya
10        maya = new Abeja();
11
12        // Vemos cuál es su estado actual
13        maya.imprimePosicionX();
14        maya.imprimePosicionY();
15        maya.imprimeEstadoVuelo();
16
17        // Ahora se va volando y se posa en otra posición del mapa
18        maya.vuela();
19        maya.imprimeEstadoVuelo();
20        maya.cambiaPosicionX(4);
21        maya.cambiaPosicionY(4);
22        maya.sePosa();
23
24        // De nuevo vemos cuál es su estado actual
25        maya.imprimePosicionX();
26        maya.imprimePosicionY();
27        maya.imprimeEstadoVuelo();
28    }
29 }
```



UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

4 - Elementos básicos del lenguaje

- Una referencia se comporta como "un mando a distancia" que nos permiten controlar un objeto desde nuestro programa.
- Al escribir la siguiente línea indicamos que nuestro programa usará una referencia capaz de controlar objetos de tipo Abeja.

```
// Definimos una referencia capaz de apuntar a objetos de la clase Abeja  
Abeja maya;
```

- Con la siguiente línea creamos un objeto usando como "molde" la clase Abeja y "vinculamos" nuestra referencia con el objeto.

```
// Ahora CREAMOS UN NUEVO OBJETO de la clase Abeja y se lo  
// asignamos a maya  
maya = new Abeja();
```



Es parecido a cuando "vinculas" tu móvil con un altavoz Bluetooth



OBJETO



REFERENCIA

UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

4 - Elementos básicos del lenguaje

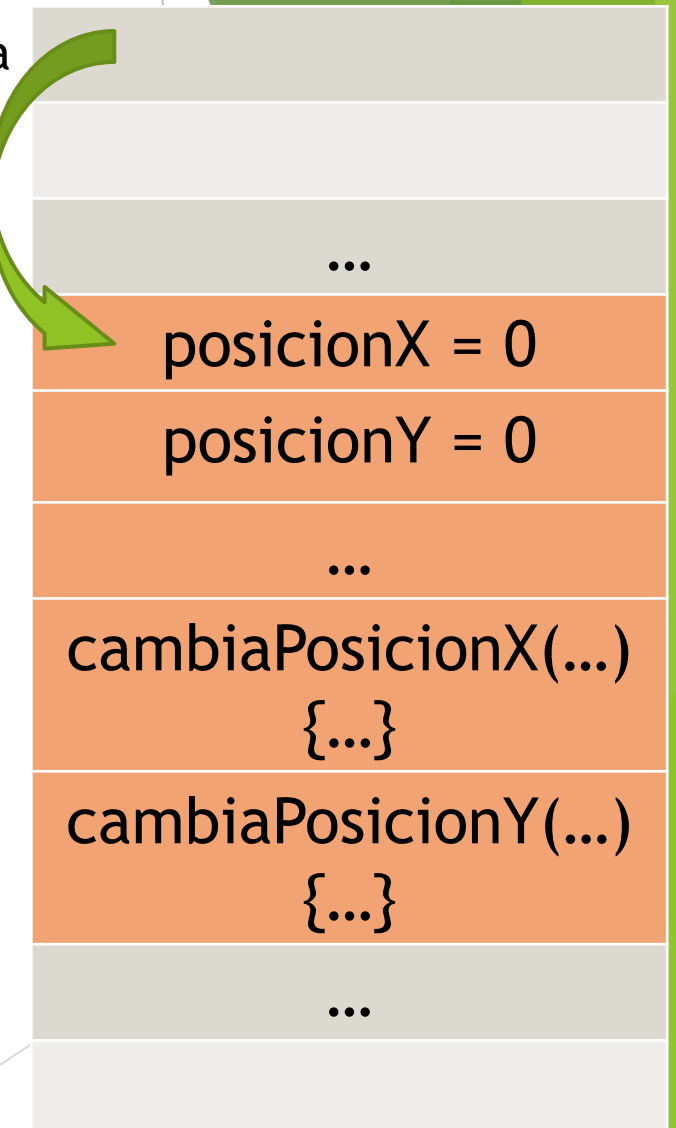
- ▶ Es importante que nos demos cuenta de qué ocurre en la memoria cuando creamos un objeto a partir de una clase.

```
// Ahora CREAMOS UN NUEVO OBJETO de la clase Abeja y se lo  
// asignamos a maya  
maya = new Abeja();
```

- ▶ Cuando hacemos el **new Abeja()** se está cogiendo todo el código de la clase (con sus propiedades y métodos) y se está copiando a una zona de memoria como si fuera una plantilla o molde.
- ▶ La dirección del nuevo objeto se le asigna a la referencia **maya** y así podremos utilizar dicho objeto desde nuestro programa.
- ▶ Además, al crear un objeto con **new** automáticamente se ponen todas las propiedades **numéricas a 0**, todos los **boolean a false** y todas las **String a "null"** (ya lo estudiaremos más adelante)

maya

Memoria



UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

4 - Elementos básicos del lenguaje

- ▶ ¡Ya hemos terminado la primera vuelta a la espiral!
- ▶ Ahora toca repasar, interiorizar y practicar todo lo que hemos visto.
- ▶ Para ello vamos a:
 - ▶ Analizar el contenido del documento “Chuleta Primera Vuelta Espiral” y preguntar todo lo que no se entienda.
 - ▶ Con este documento como referencia sintáctica vamos a realizar los siguientes ejercicios:

Haz los ejercicios del 1 a 4 del boletín de problemas



UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

4 - Elementos básicos del lenguaje

► 4.2.- Segunda vuelta a la espiral

Nos toca revisar los elementos que ya conocemos para ampliarlos y tener más “vocabulario” en nuestro lenguaje.

► *Ampliamos lo que sabemos de las propiedades*

Ahora podemos definir **varias propiedades del mismo tipo en una sola línea**. Ejemplo:

```
int edad, altura, peso;
```

```
String nombre, apellido1, apellido2;
```

```
boolean soltero, mayorEdad;
```


UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

4 - Elementos básicos del lenguaje

- *Ampliamos lo que sabemos de los métodos*
- En nuestra chuleta de la primera vuelta a la espiral tenemos lo siguiente:

Métodos	<pre>public void nombreMetodo (tipoDato1 param1, ...) { // instrucciones del método }</pre>
Instrucciones de un método	<i>Asignación:</i> propiedad = valor; <i>Imprimir en pantalla:</i> System.out.println("Texto"); System.out.println("Texto"+propiedad);

Vamos a ampliar las cosas que podemos hacer en un método

metodo

UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

4 - Elementos básicos del lenguaje

- ▶ En esta vuelta aprenderemos que en un método, además de lo anterior, podemos **definir y usar variables**.
- ▶ **¿Qué es una variable?** Es una zona de memoria identificada con un nombre en la que podemos almacenar información de nuestro programa.
- ▶ Podríamos hacer el símil de que la memoria es como una enorme estantería y una variable es una caja colocada sobre la estantería.
- ▶ La caja estaría **identificada con un nombre** y podremos hacer cosas con la caja y su contenido en cualquier momento.
- ▶ Veamos qué cosas podemos hacer con estas cajas/variables:



UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

4 - Elementos básicos del lenguaje

- **Colocar una nueva caja vacía en la estantería.** A esto se le llama “**declarar una variable**” y para ello tenemos que hacer dos cosas:
 - ▶ **Darle un nombre representativo a la caja.** Ejemplos: edad, nombre, elementoMayor, altura, color, tamaño... No vale llamarlas: ede42, wwss87...
 - ▶ **Decir qué cosas podemos meter en la caja.** En Java, si decimos que en la caja vamos a guardar zapatos, solo podremos guardar zapatos y no podremos meter otra cosa. Es decir, tenemos que especificar el tipo de dato que será capaz de guardar la variable.

Todo esto se hace en una sola línea de Java en la que se indica el tipo de datos que va a poder guardar la variable y el nombre seguido de un ;

Sintaxis de ejemplo	Significado
int edad;	Crea la variable ‘edad’ capaz de guardar números enteros
String nombre;	Crea la variable ‘nombre’ capaz de guardar cadenas de texto
double precio;	Crea la variable ‘precio’ capaz de guardar números reales

Es la misma sintaxis que se usa para las propiedades de una clase. ¡Fácil!

UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

4 - Elementos básicos del lenguaje

- También **podremos guardar un elemento dentro de una caja**, para ello usaremos el operador de asignación (igual que con las propiedades)
- Por último, también podremos **consultar su valor**, simplemente poniendo su nombre.

Ejemplo completo:

```
public class Triangulo {  
    public double longitudLado1;  
    public double longitudLado2;  
    public double longitudLado3;  
  
    public void imprimePerimetro () {  
        double perimetro;  
        perimetro = longitudLado1 + longitudLado2 + longitudLado3 ;  
        System.out.println("El perímetro del triángulo vale: "+perimetro);  
    }  
}
```

Declaramos la variable perímetro de tipo double

Guardamos un valor de tipo double

Consultamos su valor

35

perimetro

UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

4 - Elementos básicos del lenguaje

- ▶ **Las variables se utilizan muchísimo.** Sirven para guardar datos que introduzca el usuario por teclado, para realizar cálculos sobre ellas, para almacenar estadísticas...
- ▶ **Según el tipo de dato que guardan** las variables se las clasifica en dos tipos:
 - **Primitivas:** guardan tipos de datos simples: int, double, boolean, char... todo lo que no sea un objeto de una clase. Ejemplos:

char letra;

int edad;

double precio;

letra = 'A' // Es el número 65 en ASCII

edad = 33;

precio = 29.99;

Memoria	
...	
65	letra
33	edad
29.99	precio
...	



UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

4 - Elementos básicos del lenguaje

- **Referencias a objetos:** son capaces de guardar la dirección de memoria en la que está un objeto. Recuerda que cada vez que creamos un objeto de una clase con el operado **new** estamos reservando un trocito de la memoria para nuestro objeto y nuestra variable guardará la “referencia” a esa zona de la memoria.

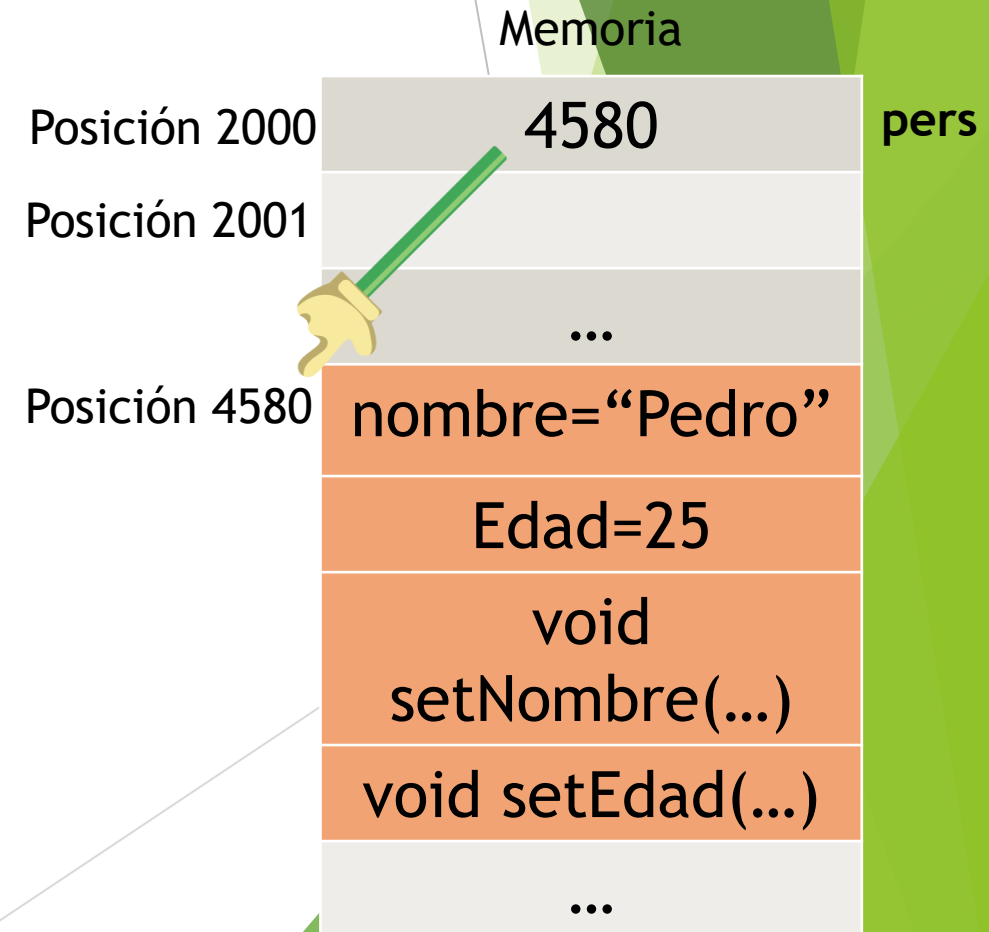
- **Ejemplo:**

```
Persona pers;  
  
pers = new Persona();  
  
pers.setNombre("Pedro");  
  
pers.setEdad(25);
```

pers es una referencia capaz de apuntar a un objeto de la clase Persona.

Funciona como un “puntero” o “apuntador”

Realiza los ejercicios 5 y 6 del boletín de problemas



UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

4 - Elementos básicos del lenguaje

Seguimos ampliando lo que sabemos de los métodos

Cuando explicamos cómo se definía un método dijimos que había un **parámetro de salida** pero que por el momento lo dejaríamos a “vacío” (void):

```
public void nombreMetodo (tipoDato param1, tipoDato param2...) {  
    ...  
}
```

← **Parámetro de salida**

Ahora vamos a completar esta definición por esta otra:

```
public tipoDatoDevuelto nombreMetodo (tipoDato param1, tipoDato param2...) { ... }
```

Ahora los métodos pueden emitir como salida un dato que hayan calculado o bien dejar la salida “vacía” (void). La jerga que se usa en estos casos es que el método “devuelve” (return) un parámetro de salida.



UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

4 - Elementos básicos del lenguaje

► Veámoslo con un ejemplo:

Fichero **Triangulo.java**

```
public class Triangulo {  
    public double longitudLado1;  
    public double longitudLado2;  
    public double longitudLado3;  
  
    public void cambiaLados (double lado1, double lado2, double lado3) {  
        longitudLado1 = lado1;  
        longitudLado2 = lado2;  
        longitudLado3 = lado3;  
    }  
  
    public double calculaPerimetro ()  
    {  
        double perimetro;  
        perimetro = longitudLado1 + longitudLado2 + longitudLado3 ;  
        return perimetro;  
    }  
}
```

TipoDevuelto

*Definimos una **variable** que nos permita guardar el cálculo realizado*

Instrucción para devolver el cálculo realizado

*Espera, espera...
Estamos hablando de
“devolver el dato calculado”
¿pero a quién?
Pues a la clase que
contenga el main*



return

UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

4 - Elementos básicos del lenguaje

- Observa el intercambio de información entre la clase *PruebaTriangulo* y el objeto de la clase *Triangulo*.

```
PruebaTriangulo.java ✕
1 public class PruebaTriangulo {
2
3     public static void main (String args[])
4     {
5         double perimetro;
6
7         Triangulo t;
8         t = new Triangulo();
9
10        t.cambiaLados(3.4, 5.0, 4.3);
11
12        perimetro = t.calculaPerimetro();
13
14        System.out.println("El perimetro del triangulo es: "+perimetro);
15    }
16 }
17
```

Definimos una variable del mismo tipo que el parámetro de salida del método `calculaPerimetro()`

La variable `perímetro` "recibe" el cálculo que nos devuelve el método `calculaPerimetro()`



PruebaTriangulo **PROPORCIONA** los parámetros de entrada a cada uno de los métodos del objeto que usa.

Y PruebaTriangulo **RECOGE** el valor devuelto por los métodos que ofrecen un parámetro de salida.



UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

4 - Elementos básicos del lenguaje

► Comparemos estos dos métodos:

```
public double calculaPerimetro () {  
    double perimetro;  
    perimetro = longitudLado1 + longitudLado2 + longitudLado3;  
    return perimetro;  
}
```

*Calcula y
devuelve el
dato calculado*

```
public void imprimePerimetro () {  
    double perimetro;  
    perimetro = longitudLado1 + longitudLado2 + longitudLado3;  
    System.out.println("El perimetro del triangulo es: "+perimetro);  
}
```

*Calcula e
imprime pero
no devuelve el
dato calculado*

Las clases
“silenciosas”
son las que se
utilizan
normalmente

Con este nuevo enfoque podemos optar entre hacer:

- **Clases “silenciosas”**: que tienen métodos que realizan un cálculo y lo devuelven pero no imprimen nada en pantalla.
- **Clases “charlatanas o ruidosas”**: que tienen métodos que realizan un cálculo y lo imprimen en la pantalla.



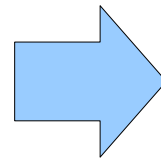
UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

4 - Elementos básicos del lenguaje

► Y, para terminar con esta espiral, revisamos las instrucciones

- Del método **main**: también podemos definir y crear un objeto en una sola línea.

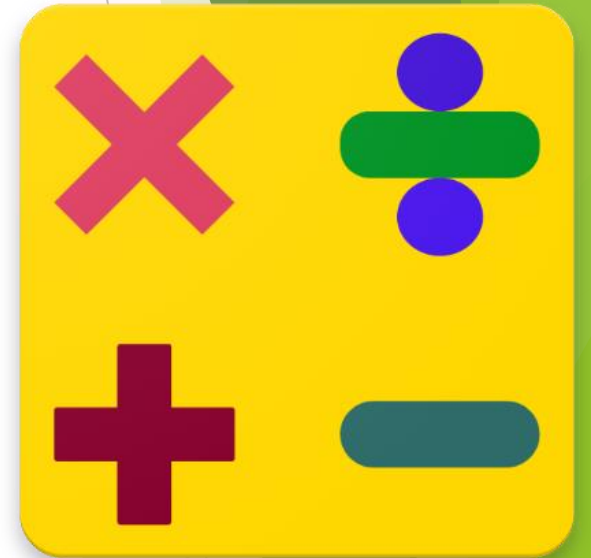
```
Triangulo t;  
t = new Triangulo();
```



```
Triangulo t = new Triangulo();
```

- Para el resto de los métodos: podemos añadir operadores aritméticos a la parte derecha de una asignación: +, -, *, / y también usar los paréntesis como en matemáticas:

```
cociente = dividendo / divisor;  
areaTriangulo = base * altura / 2;  
notaMedia = (nota1 + nota2) / 2;
```



UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

4 - Elementos básicos del lenguaje

- También podemos incrementar o decrementar en una unidad una propiedad de dos formas:

a)

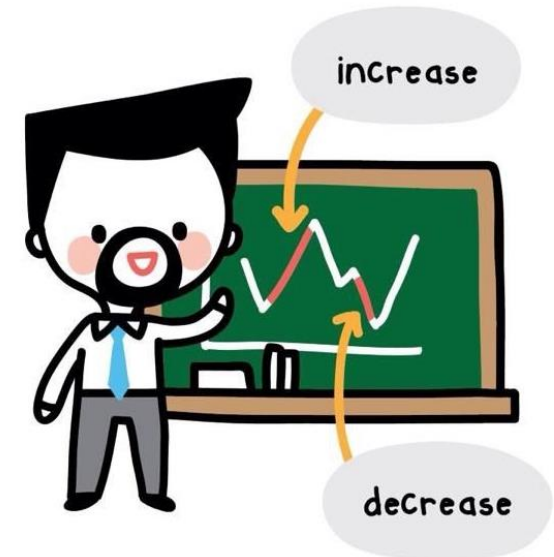
```
numCargasNectar = numCargasNectar + 1;  
numBalasDisponibles = numBalasDisponibles - 1;
```

*El resultado de este cálculo se
asignará a la propiedad de la
parte izquierda*

b)

```
numCargasNectar++;  
numBalasDisponibles--;
```

*Esta forma es más
corta y más habitual*



UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

4 - Elementos básicos del lenguaje

- Estudia a fondo la “Chuleta de la segunda vuelta a la espiral”

Haz los ejercicios del 7 al 10 del
boletín de problemas



«El que hace una pregunta puede parecer tonto durante cinco minutos,
el que no la hace será tonto toda su vida». (Proverbio chino)

UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

4 - Elementos básicos del lenguaje

► 4.3.- Tercera vuelta a la espiral

- Para terminar esta unidad didáctica daremos una vuelta más a la espiral, en la que vamos a expandir aún más nuestro vocabulario viendo los siguientes elementos:

- *Paquetes*
- *Entrada por teclado*
- *Expresiones, literales y constantes*
- *Prioridad de los operadores*



UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

4 - Elementos básicos del lenguaje

► Paquetes

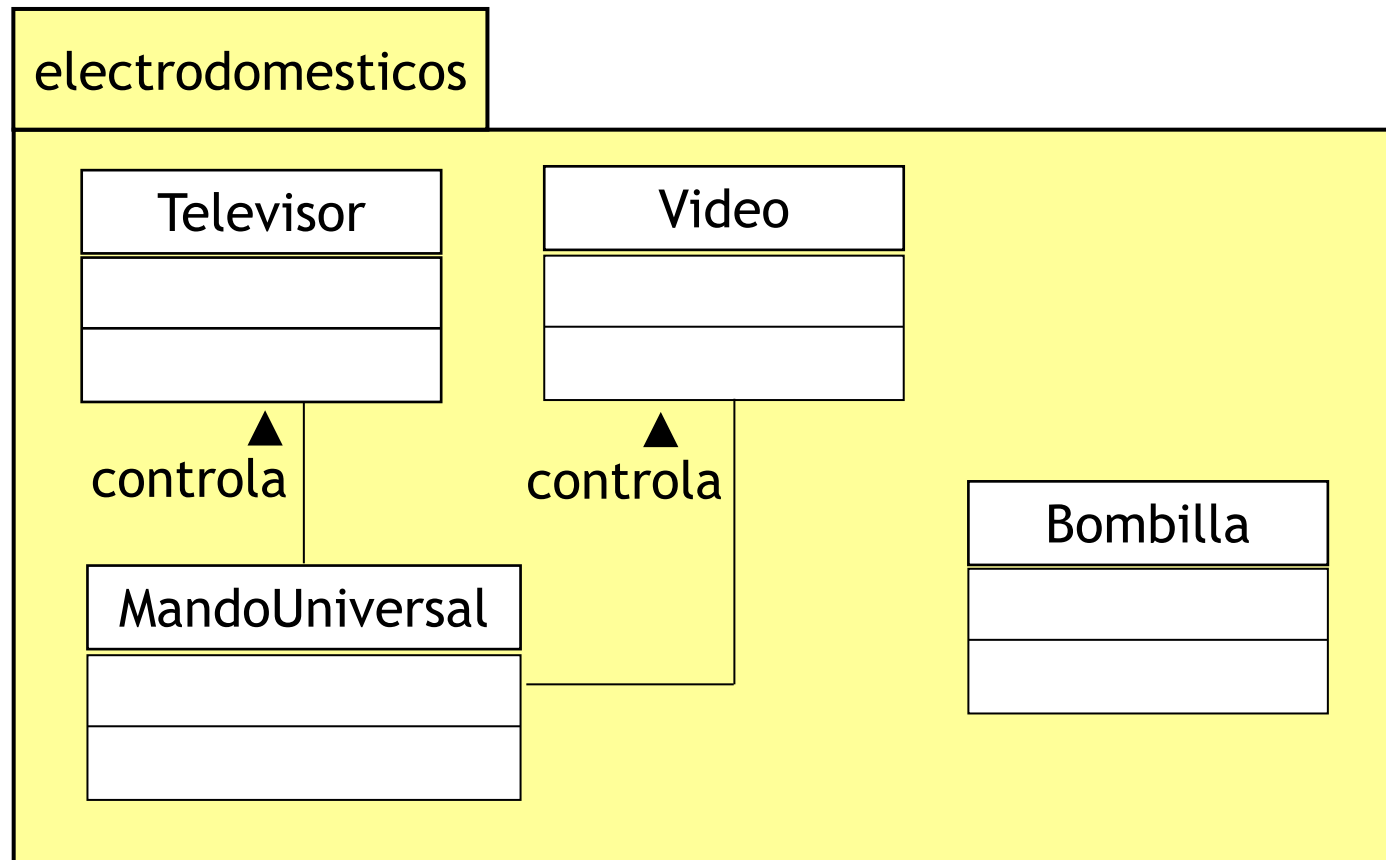
- Cuando tenemos un conjunto de clases que están relacionadas entre sí según algún criterio sería interesante poder agruparlas para poder tratarlas como un grupo.
- Por ejemplo podríamos agrupar las clases: **Televisor, Video, Bombilla y MandoUniversal** dentro de un paquete llamado **electrodomésticos** ya que podría decirse que comparten esa naturaleza.
- **El paquete es la estructura que java proporciona para agrupar y gestionar grupos de clases.** Además, un paquete también puede contener a otros paquetes.
- Podríamos asemejarlo a una caja de herramientas...



UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

4 - Elementos básicos del lenguaje

También tiene su Notación UML correspondiente. Ejemplo:



Veamos cómo se lleva todo esto al código...

UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

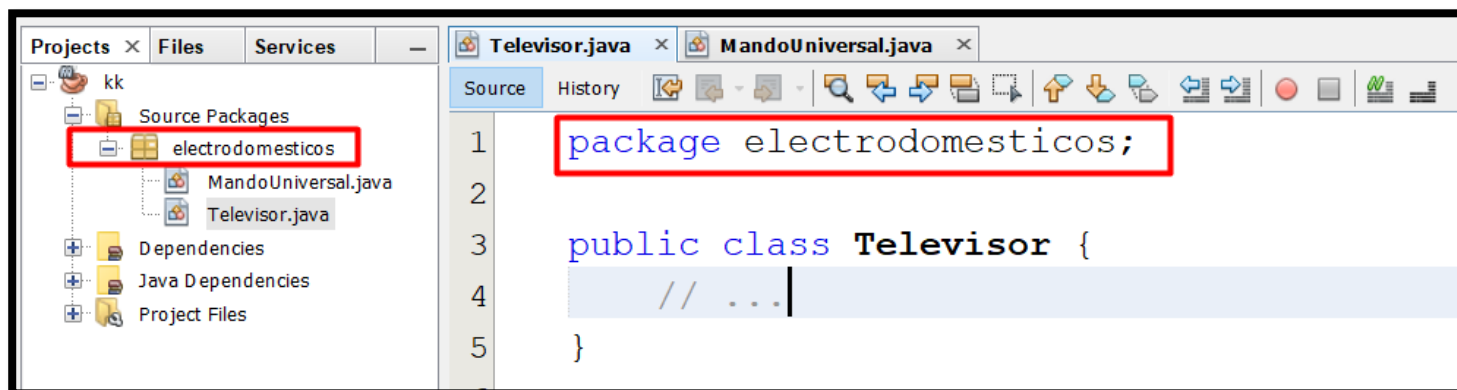
4 - Elementos básicos del lenguaje

Declaración de un paquete

- Para indicar que una clase está incluida en un paquete sólo hay que poner en la primera línea de su código fuente la sentencia:

package *nombredelpaquete;*

- Así que en el diagrama UML anterior los ficheros Bombilla.java, Televisor.java, Video.java y MandoUniversal.java deberían tener como primera línea: **package** *electrodomesticos;*



UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

4 - Elementos básicos del lenguaje

Reglas de nombrado de un paquete:

- ▶ Se escribe minúsculas y sin espacios.
- ▶ Su nombre debe comenzar con una letra
- ▶ No puede ser una palabra reservada del lenguaje (**int**, **float**, **class**...)
- ▶ Si un paquete está contenido en otro paquete, el nombre del paquete contenedor precede al nombre del paquete contenido y se separan con un punto '.'
- ▶ Ejemplos:

Nombres correctos	Nombres incorrectos
utilidades gestionclientes utilidades.graficos utilidades.sonidos	12 meses static G estion B ajas futur-soft double .number

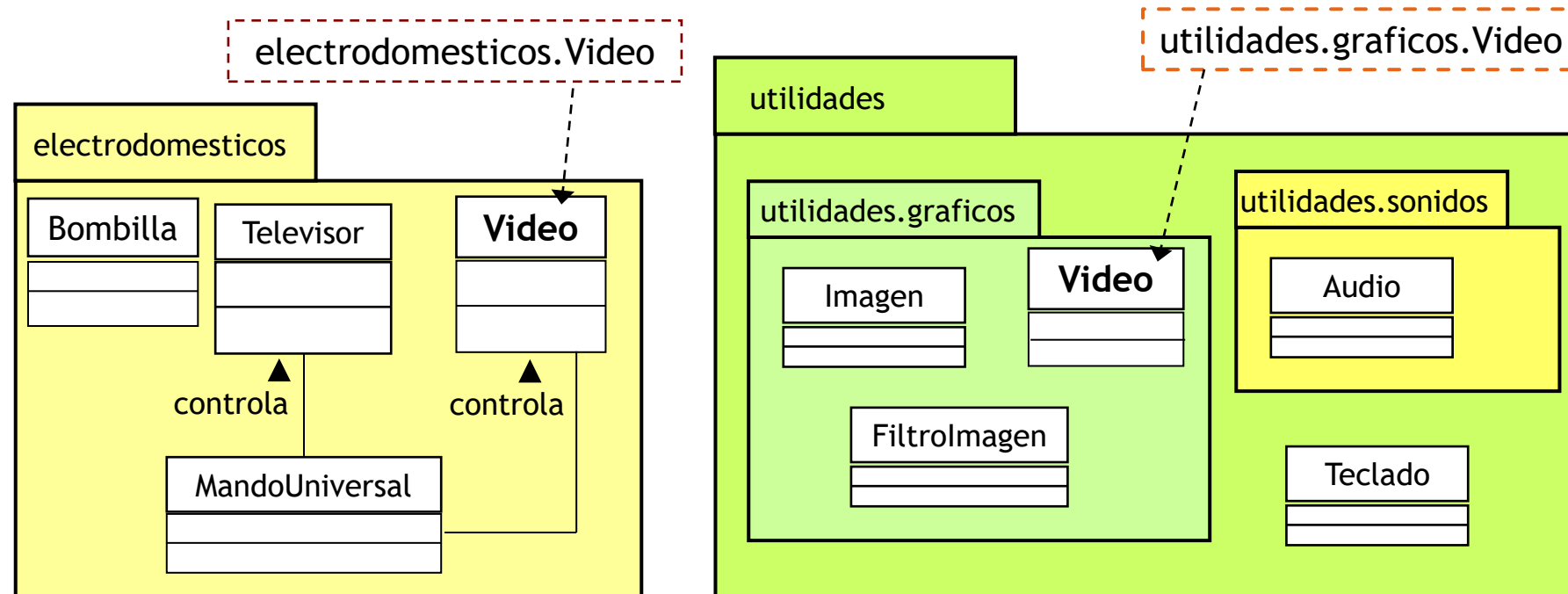


UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

4 - Elementos básicos del lenguaje

El espacio de nombres (namespace)

- Un paquete define un “espacio de nombres” que actúa como **prefijo** del nombre de las clases que contiene.
- De este modo en un mismo programa podrían existir dos clases se podría llamar igual si pertenecen a paquetes distintos. Ejemplo:



Funciona como los
prefijos de teléfono de
cada provincia

UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

4 - Elementos básicos del lenguaje

- ▶ En un mercado en el que muchas empresas desarrollando clases y se utiliza mucho software libre, **los paquetes nos ayudan a distinguir clases, resolviendo los posibles conflictos de nombrado.**
- ▶ Las empresas suelen crear una estructura de paquetes con un prefijo basado en su dirección de **dominio web pero invertida.**
- ▶ Ejemplos:
 - ▶ `www.endesa.com` → `com.endesa.servicios`
→ `com.endesa.facturacion`
 - ▶ `www.ayesa.es` → `es.ayesa.servicios`
→ `es.ayesa.facturacion`

Paquetes

De este modo, si ambas empresas colaboran en un proyecto común y comparten código siempre sabremos de qué Clase estamos hablando:

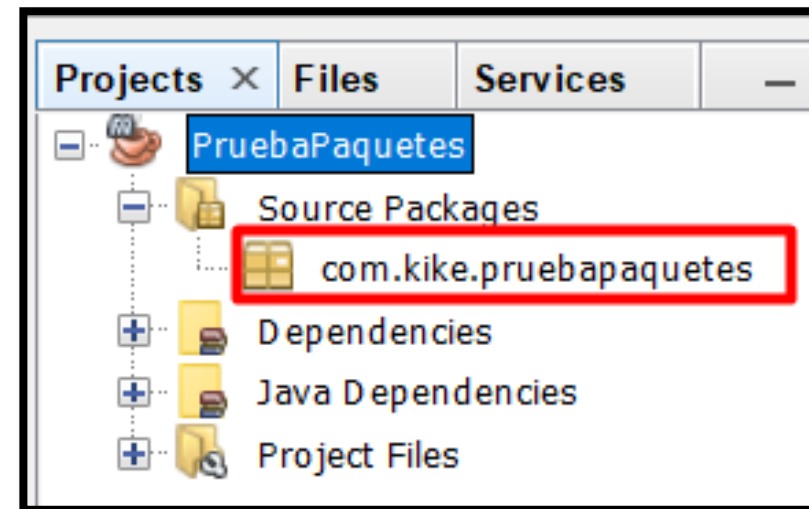
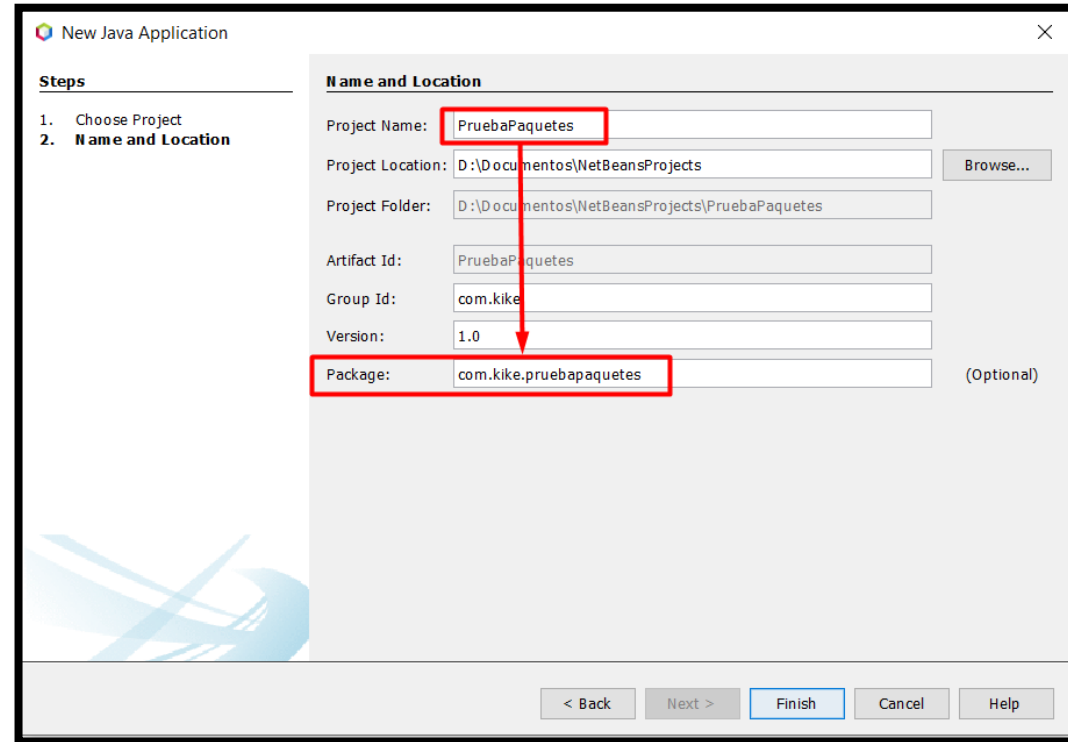
`com.endesa.facturacion.Factura` $\neq$ `es.ayesa.facturacion.Factura`



UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

4 - Elementos básicos del lenguaje

- De hecho, cada vez que NetBeans crea un proyecto con "Java Maven" ya nos lo pone en un paquete sin que nosotros hagamos nada:



- Si una clase no se declara dentro de un paquete, el compilador la añade a un paquete predeterminado o por defecto.

UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

4 - Elementos básicos del lenguaje

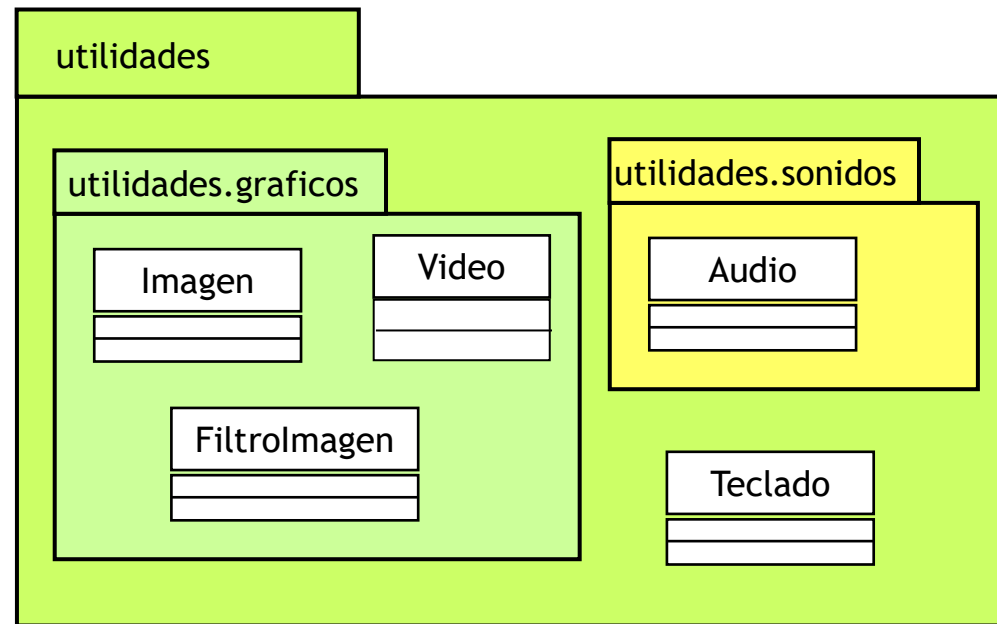
La estructura de carpetas asociada

- ▶ Sabemos que los paquetes son contenedores de clases o de otros paquetes.
- ▶ Por otro lado, los sistemas operativos organizan sus ficheros en carpetas. De modo que una carpeta es un contenedor de ficheros o de otras carpetas.
- ▶ **¿Habrá una relación entre paquete y carpeta? ¡Sí! Veámosla con un ejemplo.**



UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

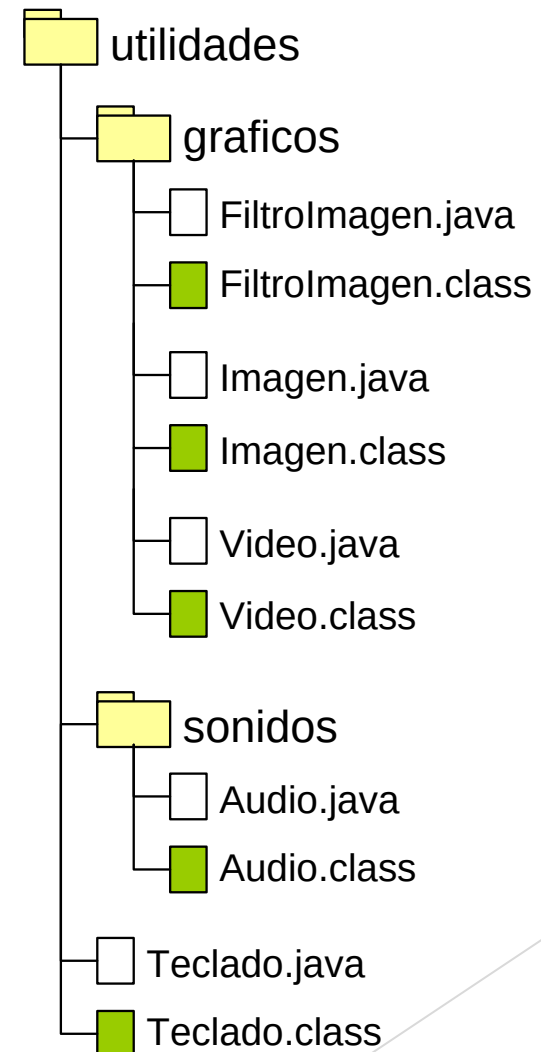
4 - Elementos básicos del lenguaje



Esta organización es la que el compilador espera. Así que si quisiéramos mover una clase de un paquete a otro tendríamos que hacer dos cosas:

- 1.- Cambiar la sentencia package y
- 2.- Mover el fichero a la carpeta que corresponda

**Si arrastramos y soltamos NetBeans
hace todo por nosotros**



**Haz el ejercicio 11 del
boletín de problemas**

UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

4 - Elementos básicos del lenguaje

La importación de paquetes

- ▶ Si una clase que está en un paquete necesita usar otra clase que está en otro paquete distinto necesitamos escribir una **sentencia de importación** para que el compilador sepa cómo incorporar esa clase a nuestro programa.
- ▶ Sintaxis de ejemplo:
 - ▶ **import** utilidades.Teclado; // Importa sólo la clase Teclado
 - ▶ **import** utilidades.graficos.*; // Importa todas las clases públicas
// del paquete utilidades.graficos



Cuando NetBeans no reconozca una clase nos avisará con una “bombillita” y nos propondrá añadir la instrucción **import** correspondiente



IMPORT
PERFECT

UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

4 - Elementos básicos del lenguaje

- ▶ Las sentencias **import** siempre se escribe justo **después de la sentencia package** (si existe).
- ▶ Se pueden importar tantas clases como se desee.
- ▶ A partir de ahora empezaremos a usar clases de la librería estándar de Java, reutilizando así lo que han hecho otros programadores.
 - ▶ Podemos ver todos los paquetes y clases de las librerías oficiales de Java aquí: <https://docs.oracle.com/javase/8/docs/api/>
 - ▶ La librería estándar contiene más de 200 paquetes que agrupan a más de 3500 clases.



Reduce

Reutiliza

Recicla

*No malgastes tu tiempo en programar lo que otro ya ha programado y compartido.
Simplemente úsalo.*

Haz el ejercicio 12 del boletín de problemas



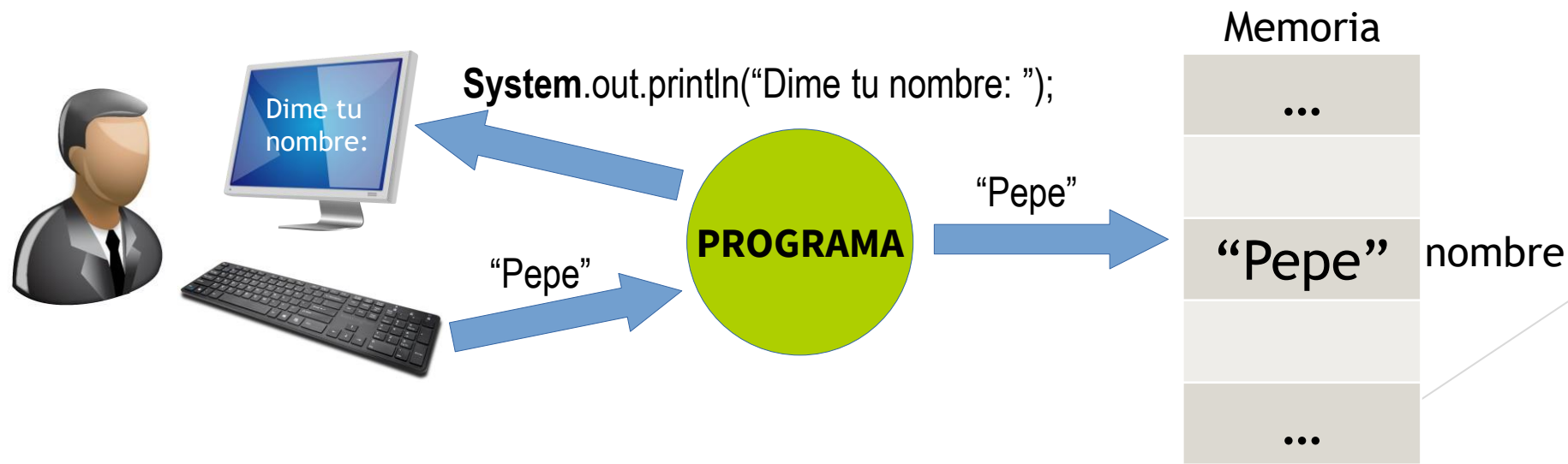
RECYCLED
SOFTWARE

UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

4 - Elementos básicos del lenguaje

Entrada por teclado

- ▶ Sabemos que con **System.out.println(...)** podemos mostrar información en la pantalla del ordenador para que la vea el usuario de nuestro programa.
- ▶ Ahora queremos que nuestro programa pueda solicitar información al usuario para que la introduzca por teclado. Y una vez que el usuario haya introducido la información, nuestro programa la guardará en una variable para poder procesarla.



UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

4 - Elementos básicos del lenguaje

Java dispone de un paquete llamado *java.util* que contiene una clase llamada **Scanner** que nos soluciona este problema. Veamos un ejemplo:

```
import java.util.Scanner;

public class ScannerDemo
{
    public static void main(String[] args)
    {
        String nombre;
        int edad;
        double salario;

        // Declaramos el objeto y lo creamos
        Scanner sc = new Scanner(System.in);

        System.out.println("Dime tu nombre: ");
        nombre = sc.nextLine();

        System.out.println("Dime tu edad: ");
        edad = sc.nextInt();

        System.out.println("Dime tu salario: ");
        salario = sc.nextDouble();

        // Imprimimos todos los valores leídos
        System.out.println("Hola "+nombre+" tienes "+edad+" años y ganas "+salario+" euros");
    }
}
```

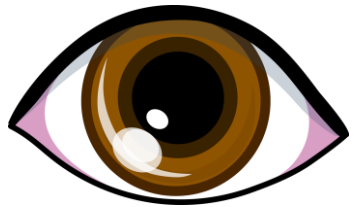


UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

4 - Elementos básicos del lenguaje

Este sería una posible entrada de datos al programa y su resultado final:

```
Dime tu nombre:
Manuel
Dime tu edad:
30
Dime tu salario:
1890,80
Hola Manuel tienes 30 años y ganas 1890.8 euros
```



¿Qué pasa si metemos un dato de un tipo distinto al que espera el programa?

```
Dime tu nombre:
Manuel
Dime tu edad:
Treinta
Exception in thread "main" java.util.InputMismatchException
    at java.base/java.util.Scanner.throwFor(Scanner.java:939)
    at java.base/java.util.Scanner.next(Scanner.java:1594)
    at java.base/java.util.Scanner.nextInt(Scanner.java:2258)
    at java.base/java.util.Scanner.nextInt(Scanner.java:2212)
    at PruebaImportaciones.main(PruebaImportaciones.java:18)
Command execution failed.
```

El programa esperaba un entero (int) y le damos un texto

UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

4 - Elementos básicos del lenguaje

El programa provoca una **Excepción**, es decir, un error que impide que la ejecución pueda continuar. Dedicaremos una unidad didáctica completa al tratamiento de excepciones... mucho más adelante.

Otra cosa interesante, si observamos la línea:

```
Scanner sc = new Scanner(System.in);
```

... hay algo raro: cada vez que hacíamos un **new** nunca pasamos parámetros de entrada. Sin embargo, ahora estamos haciendo **new Scanner(System.in)**.

Aquí tenemos que adelantar dos cosas:

- ▶ Los objetos se crean con unos métodos especiales llamados constructores que tienen el mismo nombre que la clase (Scanner en nuestro ejemplo).
- ▶ Los constructores pueden llevar parámetros de entrada, si se desea. En nuestro ejemplo **System.in** es el parámetro que le indica que queremos tomar las pulsaciones del teclado desde la entrada estándar del sistema operativo.

Haz el ejercicio 13 del
boletín de problemas

UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

4 - Elementos básicos del lenguaje

Expresiones, literales y constantes

- Cada vez que hacemos algo como:

`perimetro = lado1 + lado2 + lado3`

estamos usando una expresión en la parte derecha del =

Una **expresión** es una secuencia bien construida de operadores y elementos válidos del lenguaje.

Los operadores que conocemos son: +, -, *, /, ++, --

Estos elementos válidos pueden ser los siguientes: *literales, variables, propiedades, constantes*. Veamos cada uno de ellos:

UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

4 - Elementos básicos del lenguaje

- **Literales:** son valores concretos de un tipo de dato. Son “literalmente” lo que representan. Ejemplo: 3, 5, 8, 32.32, ‘A’, “Hola”, true, false...
- **Variables:** ya sabemos lo que son.
- **Propiedades:** ya sabemos lo que son.
- **Constantes:** son “variables” cuyo valor que no podrá ser modificado. Para declararlas se usa el modificador ***final***:

Ejemplo: **final double PI = 3.14159;**

Su nombre se suele poner todo en MAYÚSCULAS para distinguirlo de las variables normales.



4 ELEMENTS

UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

4 - Elementos básicos del lenguaje

- Para crear una expresión bien construida normalmente se realiza una secuencia del tipo: *elemento operador elemento operador elemento...*

Ejemplos:

```
perimCircunf = 2 * PI * radio;  
precioConIva = precio + precio * IVA;  
superficie = base * altura / 2;
```

+ - * y / son operadores
BINARIOS porque
necesitan DOS elementos
para realizar su operación

- Aunque también hay operadores que afectan solo a un elemento y tienen la forma: *operador elemento* o bien *elemento operador*

Ejemplos:

```
velocidad = -gravedad; // Operador para cambiar el signo  
edad++; // Autoincremento, equivale a edad = edad + 1;
```

++ -- y - son operadores
UNARIOS porque solo
necesitan UN elemento
para realizar su operación

UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

4 - Elementos básicos del lenguaje

Prioridad de los operadores

- ▶ ¿Cómo resuelve el ordenador la expresión $10 + 5 / 5$?
 - ¿En qué orden se agrupan los números y los operadores?
 - ¿Hacen falta los paréntesis para aclararlo?
- ▶ En matemáticas se estudia que la multiplicación y la división se calculan siempre antes que las sumas y las restas, entonces el ordenador haría internamente:

$$10 + 5 / 5 = 10 + 1 = 11$$

*Se calcula en
primer lugar*

- ▶ Decimos entonces que los operadores tienen un nivel de prioridad según el orden en el que se realizan los cálculos. Los que se “evalúan” primero serán más prioritarios que los que se “evalúan” después.



UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

4 - Elementos básicos del lenguaje

- De este modo podemos ordenar los operadores que conocemos actualmente de más prioritarios a menos:

+ PRIORITARIO



- PRIORITARIO

Descripción	Operadores
Paréntesis	()
operadores incremento/decremento	op++ op--
operadores unarios	+op -op
multiplicación y división	* /
suma y resta	+ -

- Esta tabla la iremos ampliando cuando aprendamos operadores nuevos



UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

4 - Elementos básicos del lenguaje

- Conocer la prioridad de los operadores nos permite escribir expresiones con más precisión sin tener que poner paréntesis “a lo loco”.
- Ejemplos:

$$\frac{a + suma * 3}{c} \longleftrightarrow (a + suma * 3) / c$$

$$j + \frac{2}{3} * (a + z) \longleftrightarrow j + 2/3 * (a + z)$$

$$\frac{a + 90}{b/c} \longleftrightarrow (a + 90) / (b/c)$$

Haz los ejercicios 14 y 15
del boletín de problemas

UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

4 - Elementos básicos del lenguaje

Cierre de la unidad

En esta unidad hemos aprendido a crear clases sencillas con sus propiedades y métodos. También hemos aprendido a crear y utilizar objetos y un montón de elementos nuevos del lenguaje.

Es mucha información nueva y rara. Tu cerebro es como tu barriga. Puedes comerte 2 kilos de carne en 10 días pero NO “el día antes de un examen”. Tu cabeza necesita tiempo para “digerir” los nuevos conceptos...

Si lo comparamos con el aprendizaje de un idioma estaríamos diciendo frases sencillas: “Dame pan”, “Gracias”, “Quiero un vaso de agua”... Si no practicas con frecuencia es fácil equivocarse y decir frases incoherentes o que no están bien construidas.



Imposible nada es.
Difícil, muchas
cosas son.

Recuerda “digerir” bien
la chuleta de la 3ª
vuelta a la espiral.
No lo dejes para
mañana.



UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA

4 - Elementos básicos del lenguaje

The background features a central cluster of colorful, semi-transparent geometric shapes (triangles and quadrilaterals) in shades of blue, green, orange, red, and purple. The text 'THE END' is prominently displayed in the center in a large, white, sans-serif font. To the right, there is a vertical strip of overlapping green and yellow geometric shapes. The overall design is modern and abstract.

THE
END